S226 PRT681/PRT585 SOFTWARE ENGINEERING: PRACTICE

Sydney Campus

Group 2:

Assignment 4: Final Group Report

TRIPMATE

Travel Planning Web Application

Prepared By:

Bibek KC (S394325) – Backend Developer and Automated Tester

Simran Chauhan (S393695) – Frontend Developer and Report Analyst

Nikesh Shrestha (S394589) – Data Analyst and Database Administrator

Susmita Sharma (S394867) – Business Analyst and Manual Tester

GitHub Repository: https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2

Date: 16 October 2026

Document control and status

This report consolidates the supplied Tripmate user manual, the Sydney Group 2 presentation, the assessment handover requirements, and the project-status details provided by the team. It is written as a handover document for.

The team has confirmed the following features as working registration and login; destination search and filtering; saved destinations; hotel booking planning records; experiences and ratings; database connection; trip and itinerary management; automated tests; and administrator add/edit/remove.

The repository URL has been supplied. Jira URL, live application URLs, exact framework/library versions, ER diagram, CSV data file, test output and screenshots have not yet.

Item	Status
Project	TripMate – travel planning web application
Team	Sydney Group 2
Frontend	React.js
Backend / data-access architecture	Presentation states .NET 8 / ASP.NET Core and Entity Framework Core; verify exact package versions from source files
Database	MySQL
API documentation	Swagger/OpenAPI is stated in the presentation; insert the actual local or hosted URL if available
Repository	https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2
Jira board	https://home.atlassian.com/o/d2706169-a840-42e2-b5e6-01ad1b2967bc/people/team/21005525-b0da-48e2-89de-d283636e83f8/about?cloudId=d3d1d3cb-d7ec-4d8d-8b7d-3091a0f54f6c
Frontend and backend URLs	Not provided

Contents

1. Project overview and objectives 5
1.1 Project summary 5
1.2 Project objectives 5
1.3 Team roles 6
1.4 Scope boundaries 6
2. Product features and user workflows 7
2.1 Registration and login 7
2.2 Destination search and filtering 8
2.3 Destination details, maps and hotels 10
2.4 Saved destinations, trip and itinerary management 11
2.5 Hotel booking planning records 12
2.6 Experiences and ratings 13
2.7 Administrator add/edit/remove destinations 14
3. Repository, tools and version inventory 16
3.1 Repository and project board 16
3.2 Technology inventory 16
4. Architecture and data flow 17
4.1 High-level architecture 17
4.2 Example data flow 17
4.3 Data model and relationships 18
5. Authentication, authorisation and security 19
5.1 Security checks 19
5.2 Privacy and responsible data handling 20
6. Git and team workflow 21
6.1 Clone and inspect the project 21
6.2 Suggested change workflow 21
7. Data analysis, database administration and data quality 22
7.1 DA/DBA responsibilities in TripMate 22
7.1.1 Verified cleaned CSV profile 22
7.2 Data cleaning and preparation 23
7.3 MySQL validation 23
7.4 Import and recovery procedure 24

7.5 ERD and relational integrity 25
8. Entity Framework Core and connection configuration 25
8.1 Package compatibility 25
8.2 Connection string and secrets 25
8.3 Entity mapping and migrations 26
9. First-time setup and troubleshooting 26
9.1 Prerequisites 26
9.2 Setup sequence 26
9.3 Common troubleshooting 27
10. Dependency injection, validation and exception handling 28
10.1 Dependency injection 28
10.2 Input validation 28
10.3 Exceptions and logging 28
11. Testing and quality assurance 28
11.1 Test levels 28
11.2 Suggested regression matrix 29
11.3 Recording defects 30
12. Guide to adding a new feature 30
12.1 Acceptance checklist 31
13. Individual contribution and evidence 31
13.1 Nikesh Shrestha – Data Analyst and DBA 31
13.2 Simran Chauhan – Frontend Developer and Report Analyst 32
13.3 Bibek KC – Backend Developer and Automated Tester 33
13.4 Susmita Sharma – Business Analyst and Manual Tester 33
13.5 Evidence register 34
13.6 Team contribution summary 32
14. Time management and team collaboration 35
14.1 Collaboration practices 35
15. Limitations, risks and future work 35
15.1 Current boundaries 35
15.2 Risks and mitigations 35
15.3 Future improvements 36
15.4 Handover notes for the next team 36

16. Handover checklist.....31

1. Project overview and objectives

1.1 Project summary

TripMate is a travel-planning web application intended to make destination discovery and trip organisation easier by bringing destination information and planning tools into one interface. The group presentation frames the problem as travel information being fragmented.

The user manual describes an Explore/Destinations experience, account creation and login, destination detail pages, filters, saved destinations, hotel planning records, user experiences and ratings, profile functions and administrator catalogue management. The user.

1.2 Project objectives

Provide a clear destination discovery experience with destination names and descriptive information.

Support search and filtering so a user can narrow the catalogue rather than browse every item manually.

Present destination information useful for planning, such as price estimates, season, trip duration, visitor estimates, ratings and suggested places where those fields exist.

Allow registered users to save destinations and maintain trip and itinerary information.

Allow hotel planning records without misrepresenting them as confirmed third-party reservations when no supplier integration exists.

Collect experiences and ratings in a way that supports traveller feedback.

Provide administrator operations for maintaining destination records and keeping catalogue data current.

Use a layered application architecture with React.js, a .NET API, a data-access layer and MySQL, as described in the group presentation.

Provide maintainable source code, automated tests, setup instructions and handover evidence.

1.3 Team roles

Member	Role	Primary handover responsibility
Bibek KC	Backend Developer and Automated Tester	API and application logic, backend integration, automated test implementation and results.
Simran Chauhan	Frontend Developer and Report Analyst	React.js user interface, frontend/backend integration, user-facing presentation and report support.
Nikesh Shrestha	Data Analyst	Destination data assessment, database structure and quality, MySQL validation, ER diagram and database documentation.
Susmita Sharma	Business Analyst and Manual Tester	Requirements, user workflows, acceptance criteria, manual testing and usability feedback.

1.4 Scope boundaries

The current confirmed scope includes the features listed in the document-control table. Live payment processing and a real hotel supplier reservation are outside the described booking-planning capability. AI-assisted chatbot or smart-search functionality is not listed as a confirmed working feature in the.

2. Product features and user workflows

2.1 Registration and login

The user manual describes visitors being able to browse destination content and registered users being able to create an account, log in and access account-related functionality. Registration and login are confirmed working by the user. A handover reader should.

Open the React.js application at the actual development or deployment URL. That URL has not been provided and must be inserted after verification.

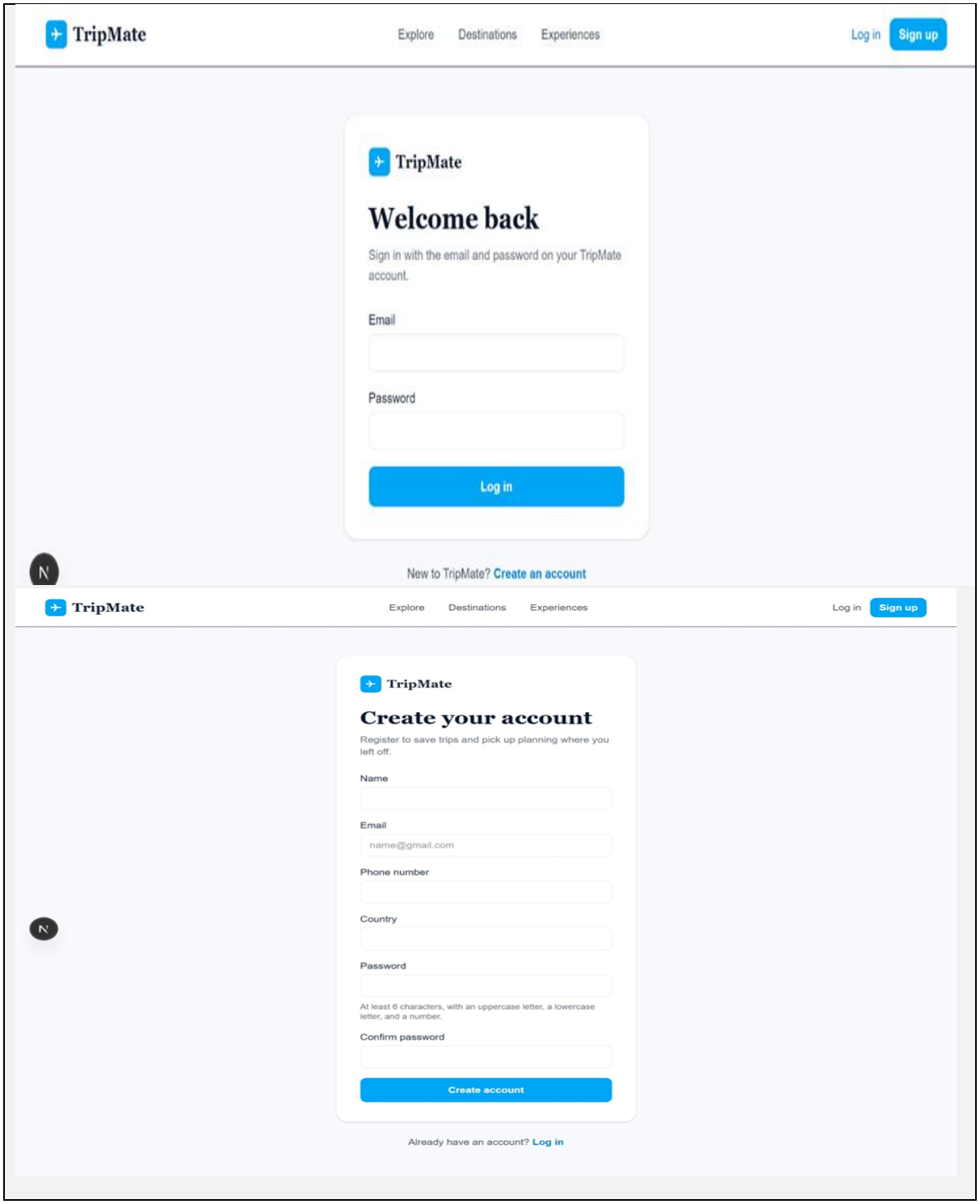
Open the registration page and enter test data only. Do not use a real password or personal information in screenshots.

Submit the form and confirm that the interface reports success or explains validation errors.

Log in with the test account, navigate to a protected feature and confirm that the user experience matches the documented rules.

Log out, if supported, and verify that protected content is no longer accessible through the ordinary user interface.

Repeat a negative test using invalid credentials and record the observed response.



2.2 Destination search and filtering

Destination search and filtering are confirmed working. The user manual describes searching by destination name, selecting a season and filtering by a minimum and maximum daily budget. It also describes sorting by visitor numbers or average daily price. The precise controls should be checked in the submitted.

Filtering is useful only when the fields used by the interface are populated and consistently represented in the database or data source. For example, if a price field contains text in some rows and a numeric value in others, comparisons.

Search: verify that entering a destination name narrows the result set as expected.

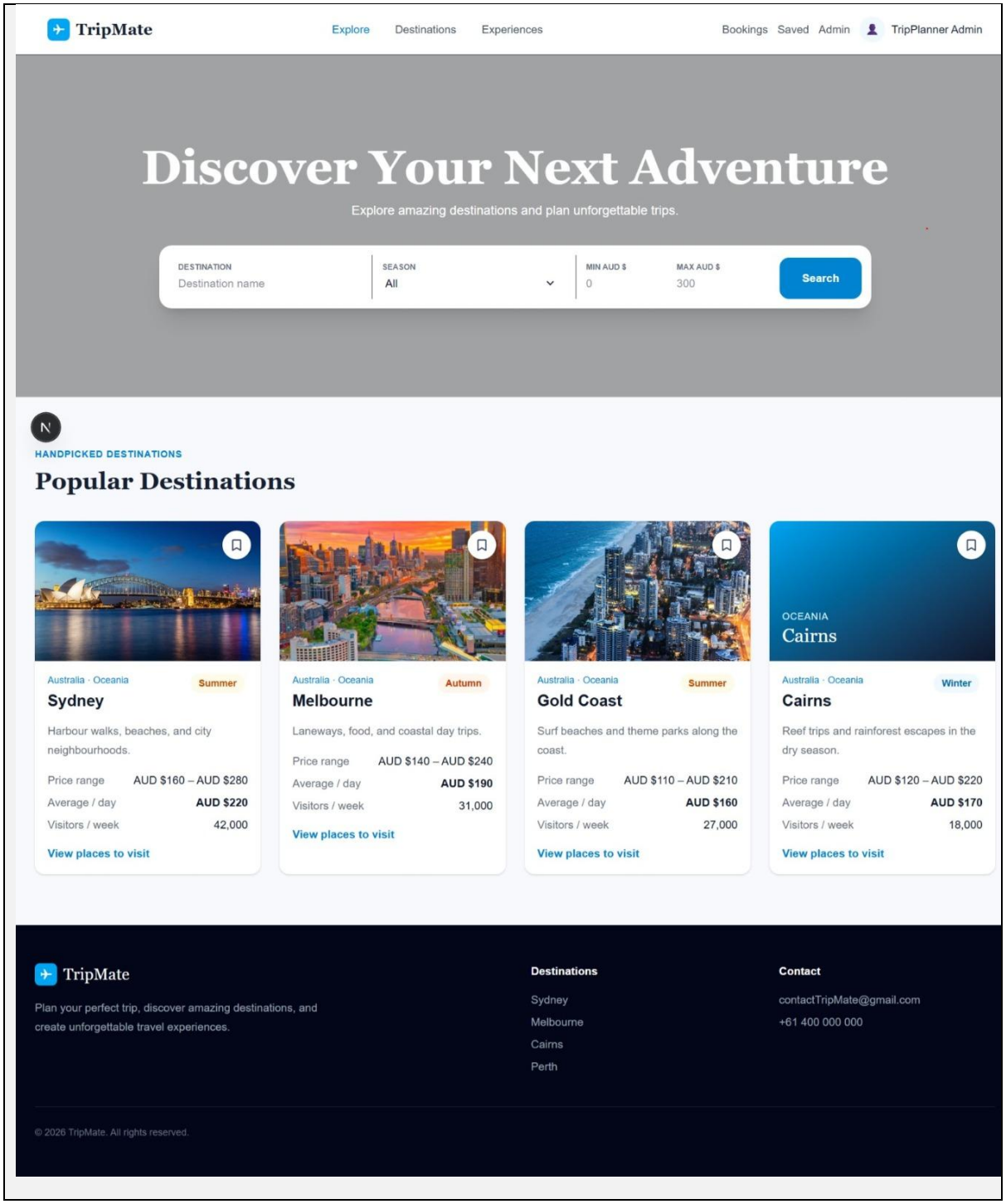
Season: test each season option and the All/reset option if present.

Budget: test the lower bound, upper bound, equal bounds and invalid range.

Sort: confirm that the chosen sort order changes the displayed order, not merely the label.

Empty state: confirm that no-match searches explain how to broaden or clear the filters.

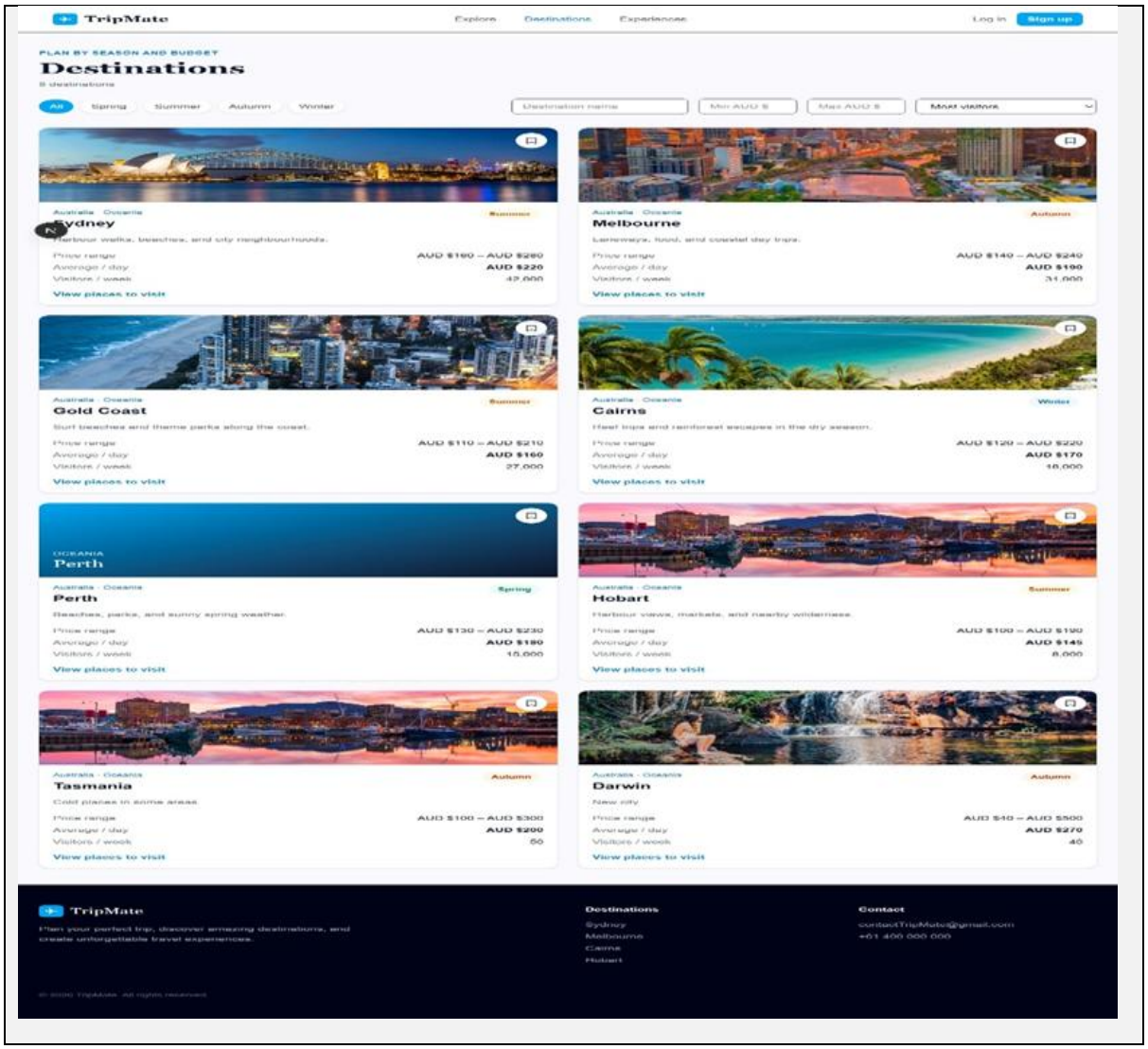
Data validation: confirm that displayed prices use a consistent currency and that the source units are documented.



2.3 Destination details, maps and hotels

The manual says a destination detail page may show the recommended season and trip duration, a description, daily price range, average daily price, visitor estimate, suggested places to visit and Google Maps links. Hotel cards may show an area, rating.

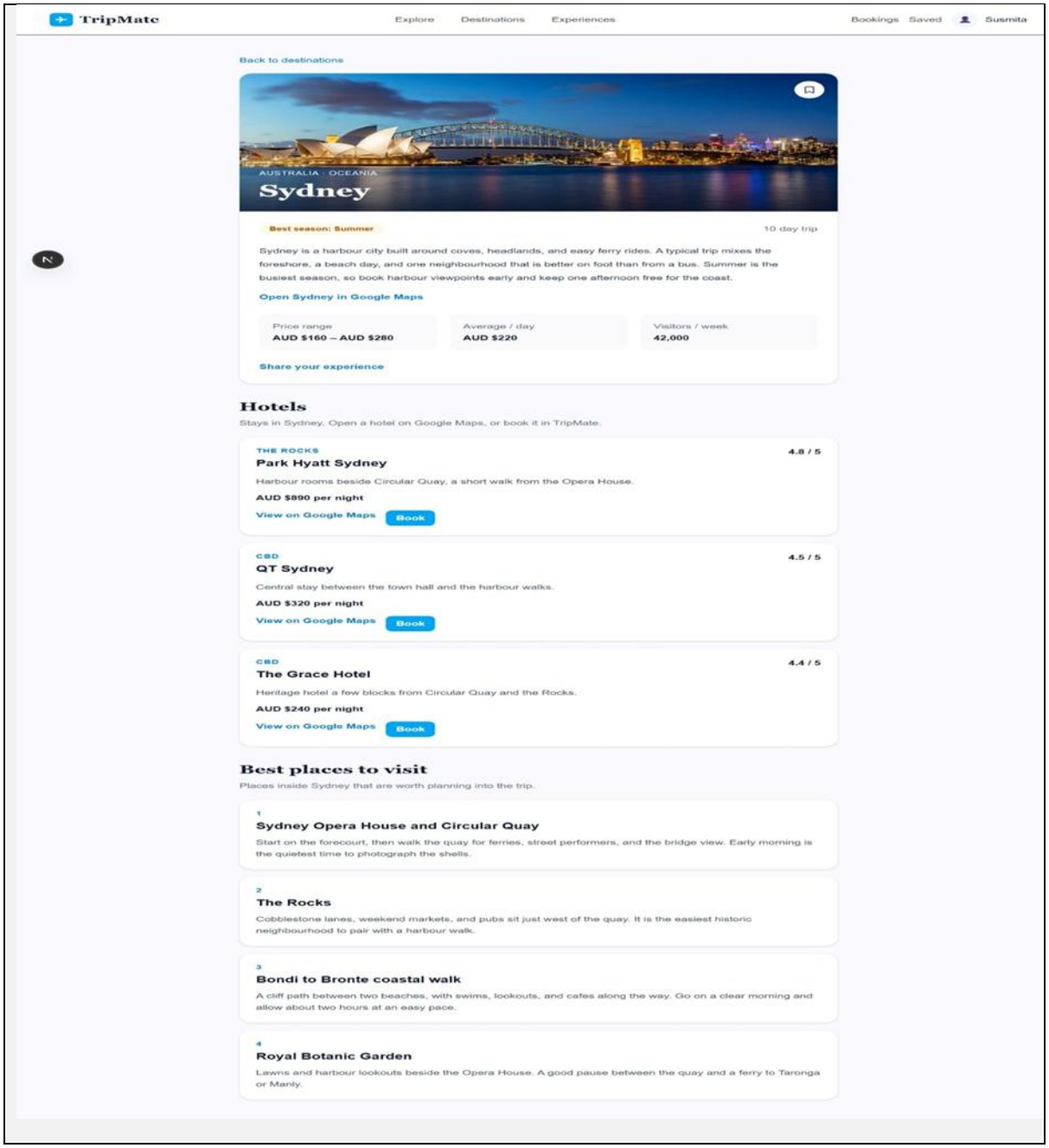
The source of every displayed value should be understandable. A destination price should be described as an estimate when it is an estimate. A missing hotel rating should not be represented.



2.4 Saved destinations, trip and itinerary management

Saved destinations and trip/itinerary management are confirmed working. These features turn discovery into an organised plan: a traveller can keep a destination for later, create a trip and arrange activities or destinations over.

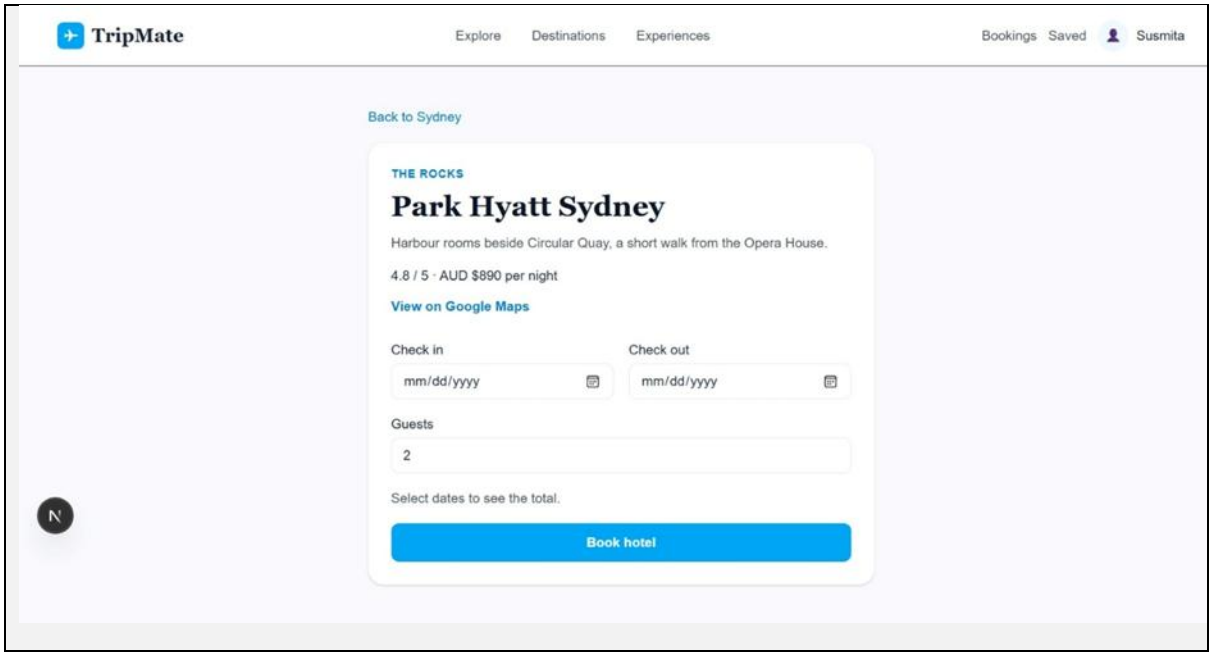
When testing, create a new test trip, add at least one itinerary entry, edit it and remove it if the interface supports removal. Check that a record belongs to the correct user and trip, and that one user's private planning data is not visible to another user unless sharing.



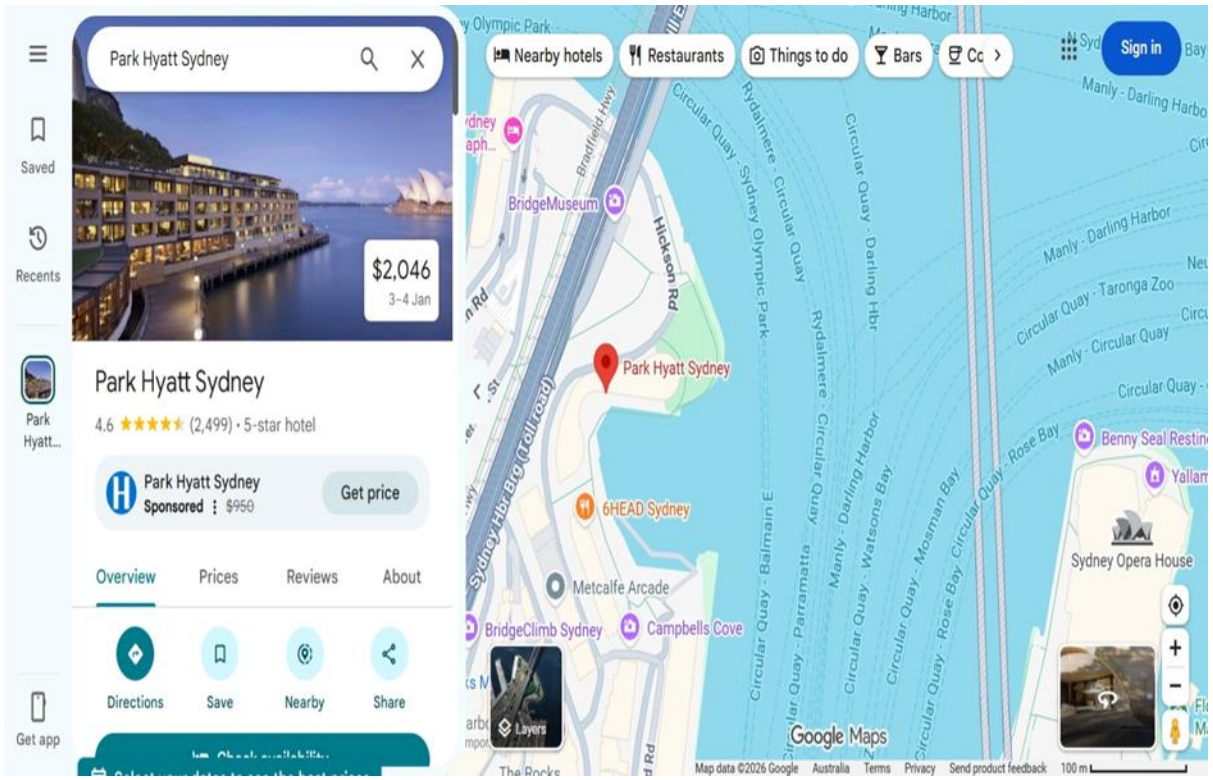
2.5 Hotel booking planning records

Hotel booking planning records are confirmed working. In this report, a planning record means that the user can record hotel-related information as part of planning. The supplied manual and presentation distinguish this from live booking/payment.

Useful tests include creating a planning record, checking the destination or hotel association, updating the record if editing is available, and checking how an empty or invalid date/price is.

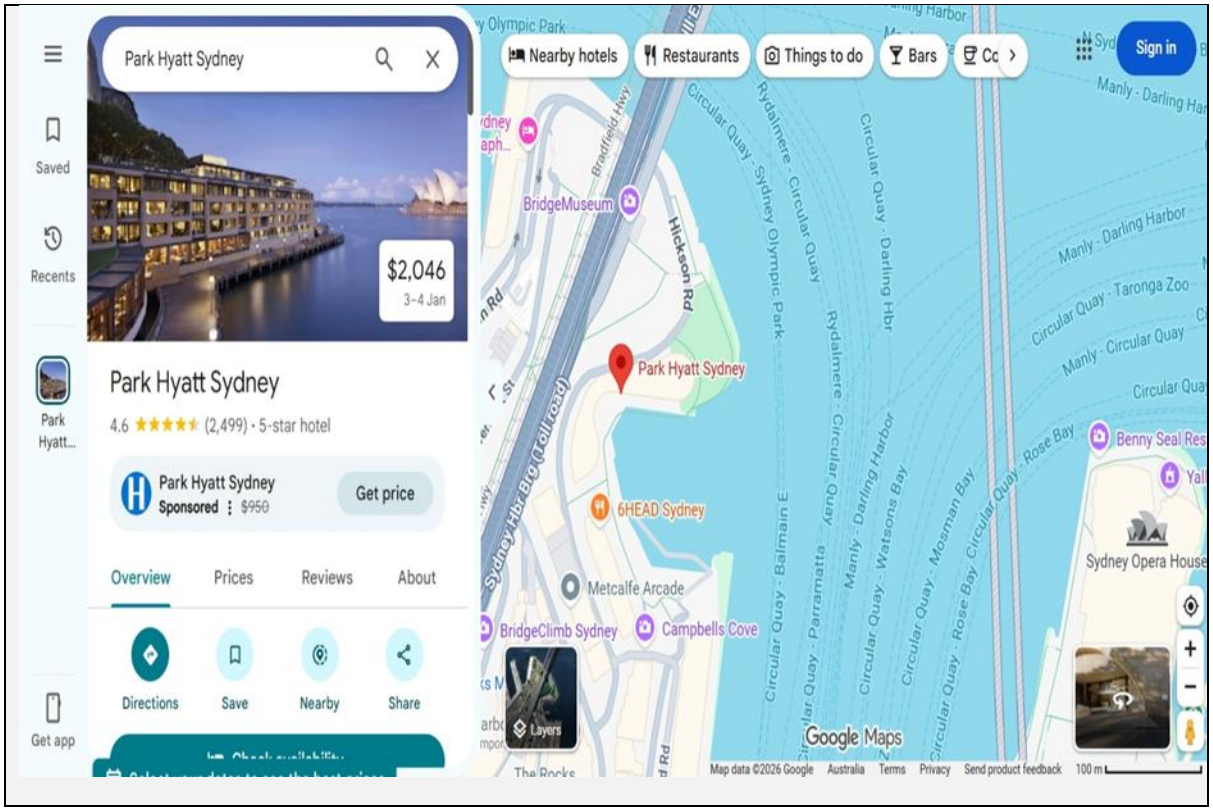


2.6 Experiences and ratings



Experiences and ratings are confirmed working. The user manual describes users sharing experiences and giving a rating, with a five-star scale. Ratings can help travellers compare destinations, but user generated.

Testing should cover submitting a valid experience, selecting a rating within the permitted range, handling empty or overly long text if relevant, and confirming that.



2.7 Administrator add/edit/remove destinations

Administrators add, edit and remove destination operations are confirmed working. These actions change shared catalogue data and should be protected from ordinary users. The user manual describes destination fields such as.

The manual describes validation expectations including required destination name, country and continent, non-negative price values, minimum price not exceeding maximum price, whole-number visitor estimate, and recommended duration from 1 to 365 days.

Sign in with a designated test administrator account.

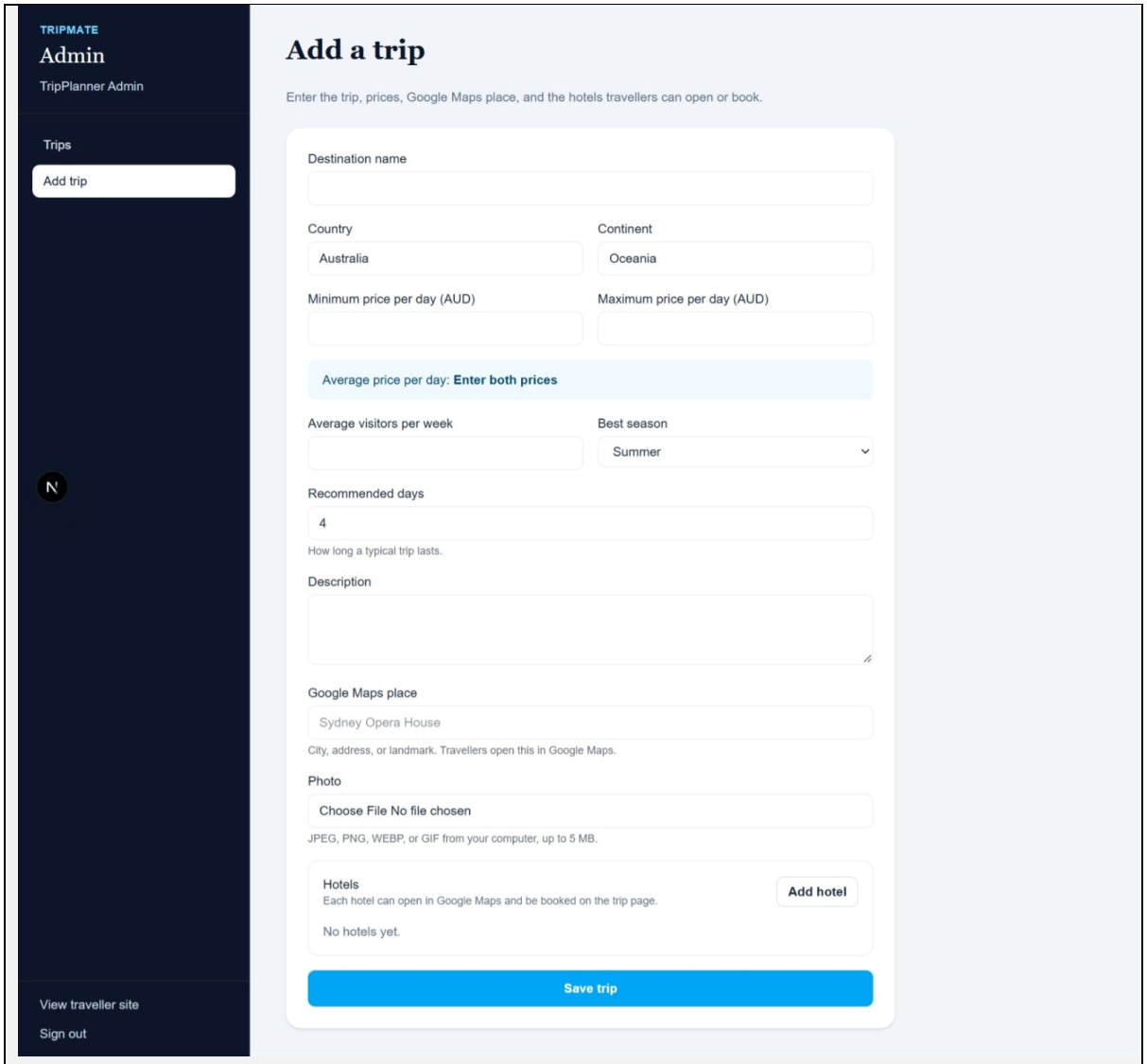
Create a destination using valid test values and confirm it appears in the catalogue.

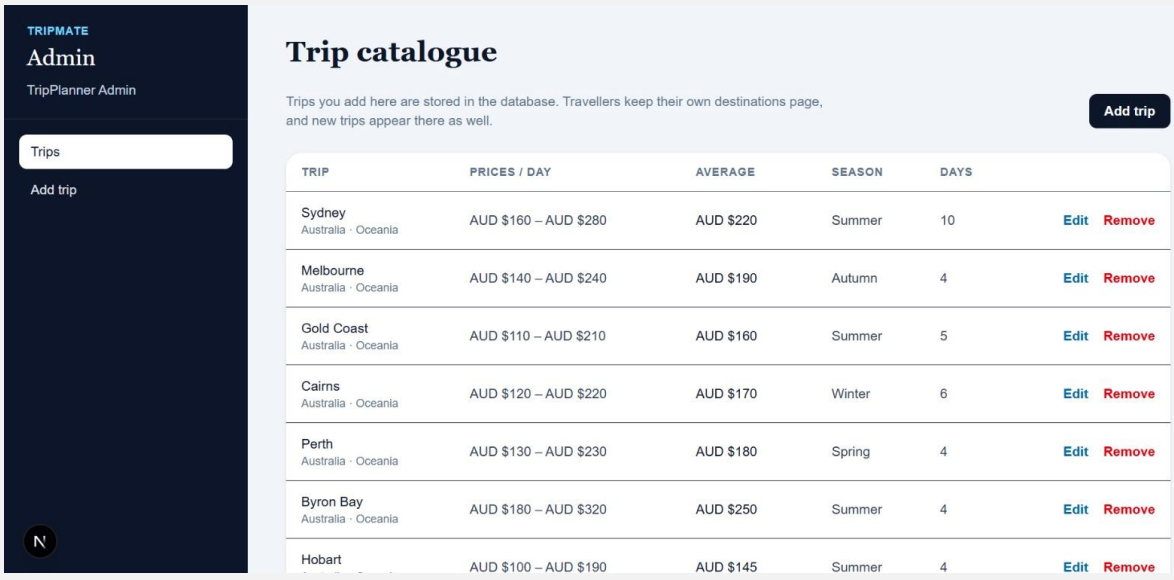
Edit a non-critical field and confirm the updated value is displayed consistently.

Attempt invalid values and record whether the system blocks the change with a clear message.

Remove a test record only after confirming it is safe to delete; use a disposable test entry rather than production data.

Repeat access checks with a normal user to verify that admin operations are not available without the required permission.


TRIPMATE
Admin
TripPlanner Admin
Trips
Add trip
View traveller site
Sign out
Add a trip
Enter the trip, prices, Google Maps place, and the hotels travellers can open or book.
Destination name
Country
Australia
Continent
Oceania
Minimum price per day (AUD)
Maximum price per day (AUD)
Average price per day: Enter both prices
Average visitors per week
Best season
Summer
Recommended days
4
How long a typical trip lasts.
Description
Google Maps place
Sydney Opera House
City, address, or landmark. Travellers open this in Google Maps.
Photo
Choose File No file chosen
JPEG, PNG, WEBP, or GIF from your computer, up to 5 MB.
Hotels
Each hotel can open in Google Maps and be booked on the trip page.
Add hotel
No hotels yet.
Save trip


TRIPMATE
Admin
TripPlanner Admin
Trips
Add trip
Trip catalogue
Trips you add here are stored in the database. Travellers keep their own destinations page, and new trips appear there as well.
Add trip
TRIP
PRICES / DAY
AVERAGE
SEASON
DAYS
Sydney
Australia · Oceania
AUD $160 – AUD $280
AUD $220
Summer
10
Edit
Remove
Melbourne
Australia · Oceania
AUD $140 – AUD $240
AUD $190
Autumn
4
Edit
Remove
Gold Coast
Australia · Oceania
AUD $110 – AUD $210
AUD $160
Summer
5
Edit
Remove
Cairns
Australia · Oceania
AUD $120 – AUD $220
AUD $170
Winter
6
Edit
Remove
Perth
Australia · Oceania
AUD $130 – AUD $230
AUD $180
Spring
4
Edit
Remove
Byron Bay
Australia · Oceania
AUD $180 – AUD $320
AUD $250
Summer
4
Edit
Remove
Hobart
AUD $100 – AUD $190
AUD $145
Summer
4
Edit
Remove

3. Repository, tools and version inventory

3.1 Repository and project board

The confirmed source repository is https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2. A handover reader should check the repository's README, folder structure, active branch, recent commits and any environment setup.

Repository access alone does not guarantee that a new developer can run the application. The repository should include safe setup instructions, dependency manifests, database scripts or migration instructions, test.

3.2 Technology inventory

Component	Known information	Evidence to insert
Frontend	React.js confirmed	package.json and lockfile; exact React and Node versions
Backend	Presentation states .NET 8 / ASP.NET Core	Backend .csproj and dotnet --info output
Data access	Presentation states Entity Framework Core	Package list and DbContext/entity files
Database	MySQL	MySQL version, schema/table list and safe connection configuration
API documentation	Swagger/OpenAPI stated in presentation	Actual Swagger URL or screenshot from local environment
Automated tests	Confirmed working by user	Test framework, test command, output and test count
Task tracking	Jira	https://sharmasusmits04.atlassian.net/jira/software/projects/SCRUM/summary

The exact versions are deliberately left open because package compatibility must be checked against the committed source files. A previously encountered EF Core/Pomelo version mismatch.

[SCREENSHOT PLACEHOLDER: Frontend and backend dependency versions]
Evidence purpose: Include package.json/lockfile and .NET package information, with no secret values.
Insert a genuine screenshot from the current application or development environment. Add figure number, date captured, and a short explanation. Remove personal data, passwords, access tokens and other secrets before inserting.

4. Architecture and data flow

4.1 High-level architecture

The group presentation describes a layered architecture: React.js provides the user interface; .NET 8 / ASP.NET Core handles API requests and application logic; Entity Framework Core is the stated data-access technology; and MySQL stores.

Layer	Responsibility	Evidence to verify
React.js UI	Renders pages, forms, destination cards, filters, saved items, trip/itinerary views and admin interfaces.	Frontend source tree, package.json, browser screenshots.
ASP.NET Core API	Receives HTTP requests, validates input, applies rules and returns responses.	Controllers or endpoint definitions, Program.cs, Swagger.
Authentication and authorisation	Supports registration/login and controls protected operations according to actual implementation.	Identity/JWT/session configuration and access tests.
EF Core data access	Maps application entities to database operations if used in the current build.	DbContext, entity classes, provider package and migrations.
MySQL	Stores structured records according to the actual schema.	ER diagram, SQL scripts, table definitions and sample queries.

4.2 Example data flow

A traveller enters a destination name or chooses a filter in the React.js interface.

The frontend applies any immediate user-interface validation and sends the request to the actual documented API route.

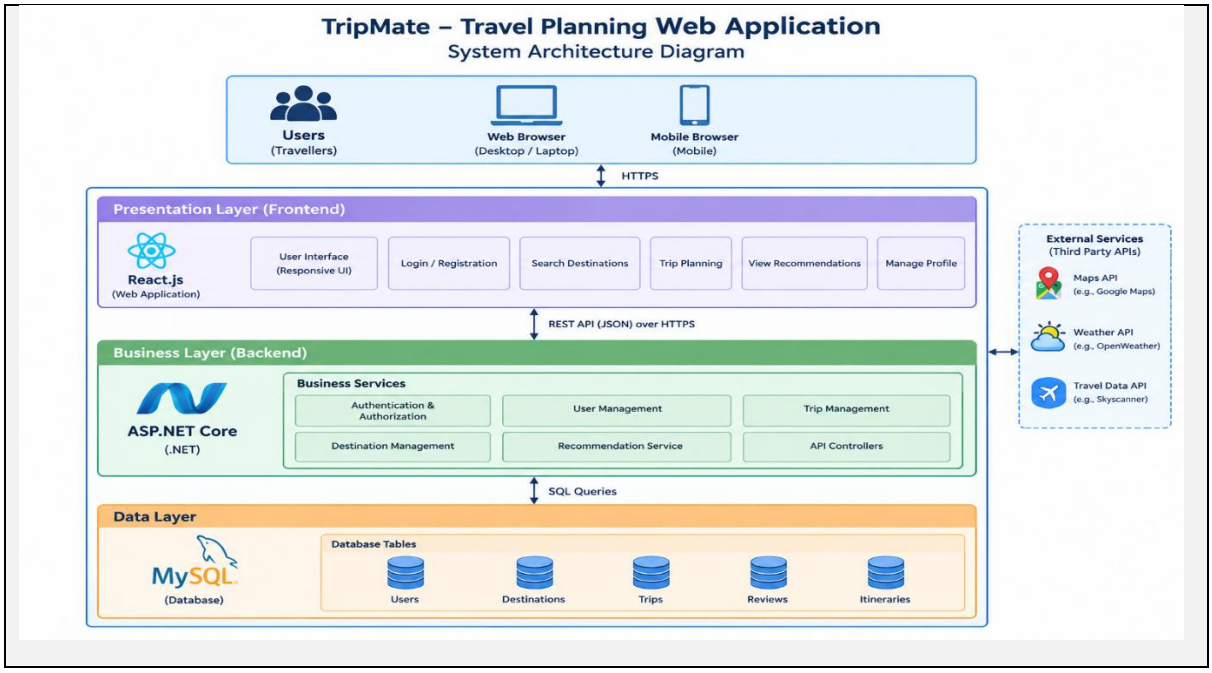
The backend receives the request and validates its parameters.

The application logic queries the data-access layer; where EF Core is used, the context maps the query to the MySQL database.

MySQL returns matching records, and the backend serialises the response.

The frontend renders the returned results or an appropriate empty/error state.

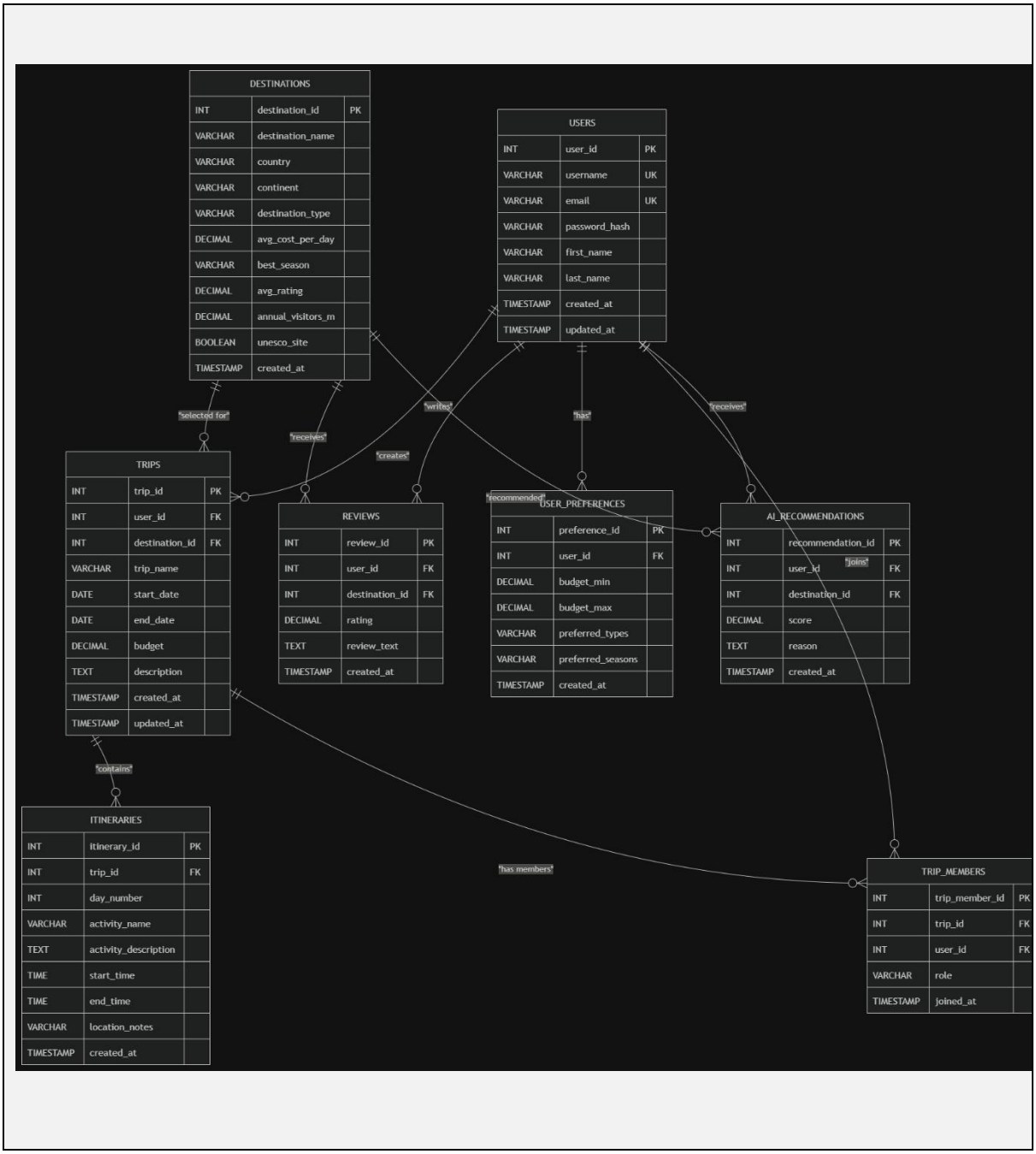
This sequence is a conceptual explanation of the architecture described in the presentation. It is not a claim about the exact controller names, route names or.



4.3 Data model and relationships

The ER diagram is an essential handover artefact because it documents the entities and relationships that the application uses. It should show table names, primary keys, foreign keys, relationship cardinality.

The team-provided ER diagram is inserted below. It documents the proposed database entities and relationships; verify it against the current MySQL schema before submission.



5. Authentication, authorisation and security

Registration and login are confirmed working, but the exact authentication mechanism must be confirmed from the code. The group presentation mentions ASP.NET Identity and JWT. The final report should.

5.1 Security checks

Use a test account and never include real passwords in screenshots or documentation.

Check that failed login attempts produce a safe, understandable response.

Verify that protected endpoints validate the authentication mechanism used by the project.

Verify that only an authorised administrator can add, edit or remove shared destinations.

Check that a user cannot read or modify another user's private saved items, trip records or experiences unless sharing is intended.

Keep database credentials and signing keys in environment variables or a suitable local secret store; exclude secret-bearing files from version control.

Use parameterised queries or ORM-generated parameterised operations rather than concatenating untrusted input into SQL.

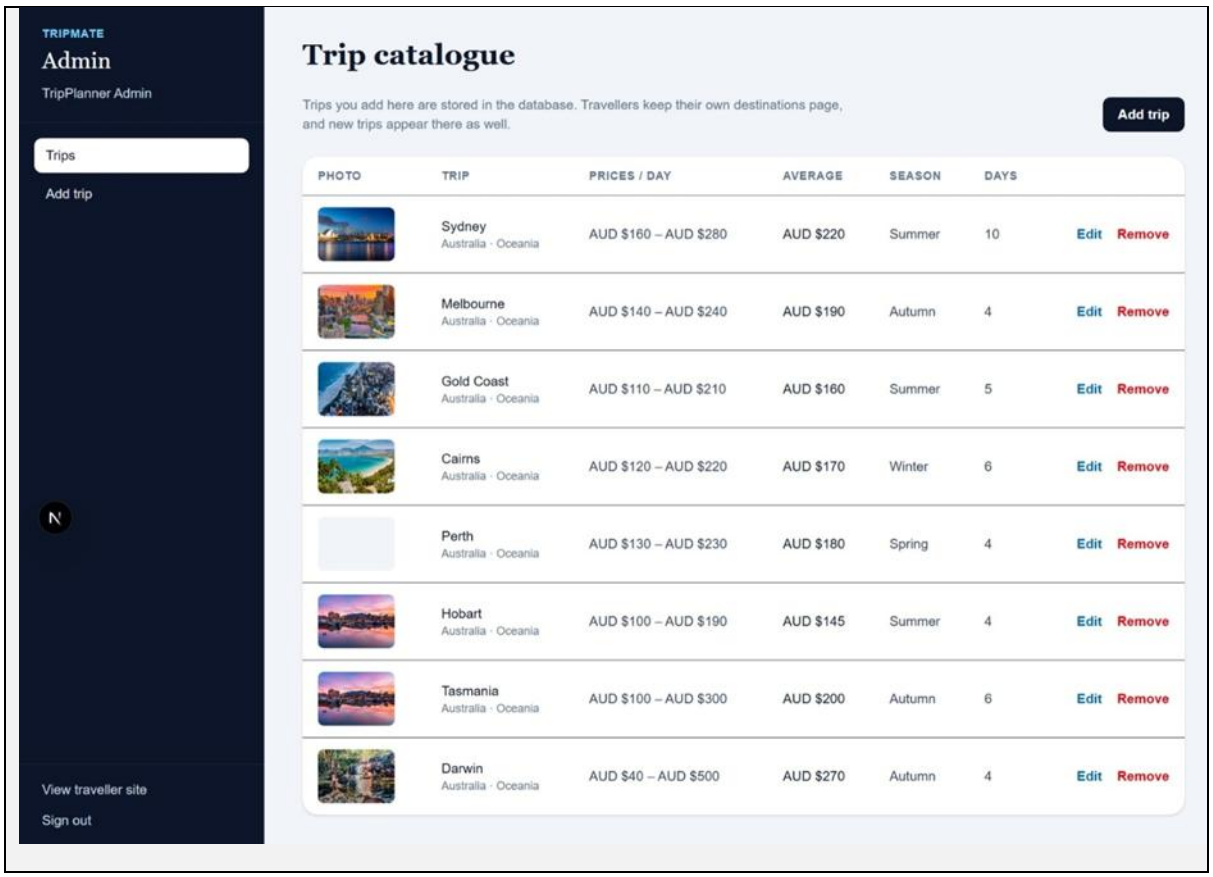
Validate data on the server, even if the frontend also validates it.

Return useful error messages without exposing stack traces, connection strings, internal paths or database details to end users.

Review dependency updates and known vulnerabilities before deployment.

5.2 Privacy and responsible data handling

Trip planning can involve account information, personal preferences and user-generated experiences. The application should collect only the data needed for its stated features, explain how the data is used, and avoid placing personal data in public.



6. Git and team workflow

6.1 Clone and inspect the project

Use the repository URL supplied above. The actual folder names and startup commands must be read from the repository; they are not assumed in this report. A typical initial workflow is:

```
git clone https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2
cd S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2
git status
git branch --show-current
```

If repository access is private, the developer must authenticate through GitHub's approved method. Do not paste access tokens into the report, terminal screenshots or shared messages.

6.2 Suggested change workflow

Clarify the requirement and acceptance criteria with the relevant role owner.

Check the task board or agreed team tracking method and confirm that the work is not duplicated.

Pull the agreed base branch and create a focused branch for the change.

Inspect the existing UI, API, data model and tests before editing.

Implement the smallest coherent change and keep naming consistent with the project.

Run relevant automated tests and build commands; test the user journey manually where appropriate.

Review the diff to remove unrelated changes, generated output and secrets.

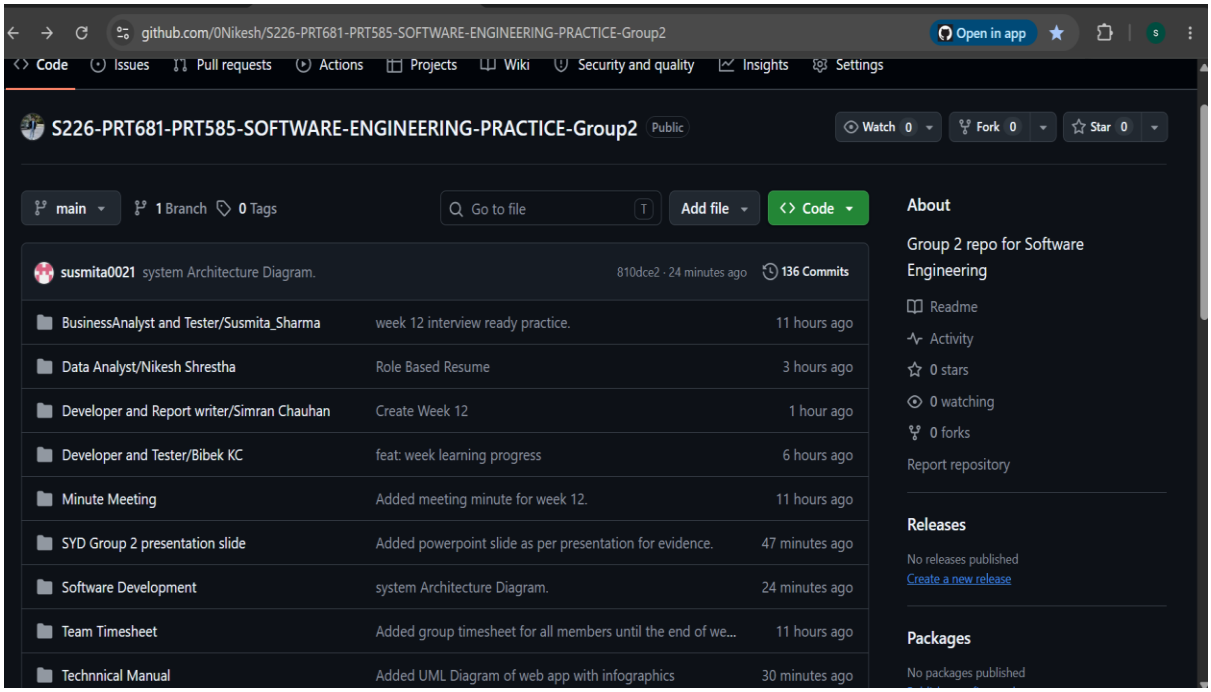
Commit with a descriptive message and push the branch.

Request review, address feedback and merge according to the team's process.

Update the task status, test evidence, setup notes and handover documentation.

The repository link :
https://sharmasusmits04.atlassian.net/jira/software/projects/SCRUM/summary

Jira board URL:
https://sharmasusmits04.atlassian.net/jira/software/projects/SCRUM/summary



7. Data analysis, database administration and data quality

7.1 DA/DBA responsibilities in TripMate

The DA/DBA contribution focuses on the reliability and organisation of the information that the application uses. Data analysis includes understanding the source dataset, checking which fields support the product requirements.

The cleaned destination CSV has now been supplied and is profiled in Section 7.1.1. The original raw dataset and cleaning history have not yet been provided.

7.1.1 Verified cleaned CSV profile

The user supplied Australian_Tourist_Destinations(1).csv as the cleaned dataset. Inspection of this file confirms 82 rows and nine columns. All 82 rows have a value in every column, and there are zero fully identical duplicate rows. There are 55 distinct destination names, so repeated names warrant review with other fields rather than automatic deletion. These observations describe the supplied cleaned file only; the original uncleaned dataset and a cleaning log were not supplied, so specific transformations cannot be verified.

CSV column inventory: Destination Name (text); Country (text); Continent (text); Type (text); Avg Cost (USD/day) (numeric); Best Season (text); Avg Rating (numeric); Annual Visitors (M) (numeric); UNESCO Site (0/1 numeric flag). Country and Continent each have one unique value in this file: Australia and Oceania respectively. Type has six distinct values and Best Season has four.

Data-to-ERD mapping for review: Destination Name → DESTINATIONS.destination_name; Country → country; Continent → continent; Type → destination_type; Avg Cost (USD/day)

→ avg_cost_per_day; Best Season → best_season; Avg Rating → avg_rating; Annual Visitors (M) → annual_visitors_m; UNESCO Site → unesco_site. The ER diagram also includes destination_id and created_at, which are not CSV columns. The actual import mapping and database-generated fields must be checked against the MySQL schema and import script. Importantly, the CSV labels costs in USD/day: do not describe these values as AUD/day unless a documented conversion was applied.

This profile verifies the condition of the cleaned dataset, not the exact cleaning operations personally performed. To complete the methodology, supply the original CSV or a short description of which cleaning steps you applied, and a MySQL row-count screenshot or query result to verify the database import.

7.2 Data cleaning and preparation

A responsible data-preparation workflow begins by preserving an unchanged copy of the source file. Record the source, download date, licence/attribution information and any restrictions. Next, inspect the header row, data types.

The following checks are appropriate to consider, but the final report must record only those that were performed:

Missing or blank values in fields needed for destination search and detail pages.

Duplicate candidates based on destination name and location; do not delete rows based on name alone if two different places can share a name.

Inconsistent whitespace, capitalisation, spelling and country/region labels.

Invalid numeric values, such as a negative price or rating outside the intended scale.

Different units or currencies in price fields; do not compare amounts until units are consistent.

Date or season values that do not match the options supported by the application.

Fields imported as text when the application expects a numeric or date type.

Rows that are valid in the source but outside the scope of the TripMate catalogue.

Cleaning should be traceable. Keep a note of the rule applied, number of rows affected, and whether the source row was corrected, excluded or left unchanged.

7.3 MySQL validation

After importing the CSV or inserting records, compare the expected row count with the database count. Verify that the import reached the intended database and table; a successful import.

The SQL below is illustrative only. Run it only after replacing the database, table and column names with those confirmed by the ERD and MySQL schema.

```
USE [actual_database_name];
SHOW TABLES;
DESCRIBE [actual_table_name];
SELECT COUNT(*) FROM [actual_table_name];
SELECT * FROM [actual_table_name] LIMIT 10;
```

For MySQL, square-bracket identifiers are not the normal identifier syntax; substitute actual names without square brackets, or use backticks for identifiers that require quoting. Example duplicate review, after adapting the names:

```
SELECT destination_name, country, COUNT(*) AS occurrences
FROM destinations
GROUP BY destination_name, country
HAVING COUNT(*) > 1;
```

Example missing-name review:

```
SELECT * FROM destinations
WHERE destination_name IS NULL OR TRIM (destination_name) = '';
```

Example price-range review, if the schema has separate minimum and maximum columns:

```
SELECT * FROM destinations
WHERE min_price < 0 OR max_price < 0 OR min_price > max_price;
```

These queries are not proof that the project uses a table called destinations or fields called destination_name, min_price and max_price. Adapt them.

7.4 Import and recovery procedure

Back up the current development database or use a disposable development schema before importing.

Confirm the database name and table name in the SQL client or import wizard.

Check the CSV delimiter, character encoding, header-row setting and column mapping.

Run the import once, then inspect the import summary and actual table row count.

Query sample rows and compare them with the source CSV.

Run duplicate, null and domain-validation checks adapted to the actual schema.

If the data was imported into the wrong table, document the issue and plan a controlled correction; do not delete tables or records until a backup and dependency check are complete.

If duplicate import rows exist, identify the exact affected records and use a safe, reviewed correction script rather than deleting records solely by a broad name match.

Record the final database name, table list, import date, row count and known data limitations.

7.5 ERD and relational integrity

The ER diagram should reflect the actual MySQL schema. Each table should have a clear purpose and a suitable primary key. Foreign keys should connect records that depend on one another, such as itinerary entries.

The ERD should also distinguish server-side tables from data stored in browser storage. The supplied manual says some saved destinations, hotel planning records and experiences may be held in browser.



Id	Name	Country	UserName	NormalizedUserName	Email
0576fa5c-49e3-4e16-9c2f-5f92d9dcb217	Hello	USA	hello@gmail.com	HELLO@GMAIL.COM	hello@gmail.c
13f84b6e-5ee8-4578-8312-65fb4cc2566c	Simran	India	simran@gmail.com	SIMRAN@GMAIL.COM	simran@gmai
26b6ad0d-bb7c-49fb-a64e-c496b5b74912	Bibek	Nepal	bibek@gmail.com	BIBEK@GMAIL.COM	bibek@gmail.(
4545f416-4004-4479-9877-b6ad22860197	TripPlanner Admin	Australia	admin@tripplanner.com	ADMIN@TRIPPLANNER.COM	admin@trippla
5dfb290e-780b-44e8-be31-87a914901ddb	Test	Afganistan	test@gmail.com	TEST@GMAIL.COM	test@gmail.co
b0843846-3805-4874-83cb-85a6ae4f3476	Susmita	Nepal	susmita@gmail.com	SUSMITA@GMAIL.COM	susmita@gma

8. Entity Framework Core and connection configuration

The group presentation states that Entity Framework Core is used for data access between the .NET backend and MySQL. A new developer should locate the DbContext, entity classes, provider package and.

8.1 Package compatibility

EF Core and the MySQL provider must be compatible. The team previously encountered a package-version mismatch during development; therefore the final versions

dotnet --info
dotnet list package
dotnet restore
dotnet build

If EF tooling is installed and migrations are part of the workflow, also inspect the tooling version and migration status using the commands.

8.2 Connection string and secrets

Connection configuration should be local to the developer environment or supplied through a safe configuration mechanism. The report may show the connection-string shape but must never include a real password:

Server=localhost;Database=TripMate;User ID=[local_user]; Password=[local_password];

To make app secure we are not uploading .env file in github as env contains the confidential logins for admin and database. To access env please contact us.

8.3 Entity mapping and migrations

Entity properties should match the database types and nullability. A numeric price should not be mapped as arbitrary text if it is used in comparisons; a rating should have a defined scale; and primary and foreign keys should.

9. First-time setup and troubleshooting

9.1 Prerequisites

Git and access to the supplied repository.

Node.js and npm versions compatible with the frontend package files.

.NET SDK compatible with the backend target framework stated in the project.

MySQL Server and a safe local development account.

An editor or IDE appropriate to the team's project.

Any required local environment variables and database scripts.

Access to the task board or the team's agreed progress-tracking method.

9.2 Setup sequence

Clone the repository and inspect the README before running commands.

Locate the frontend and backend folders from the actual repository structure.

Check Node.js, npm and .NET versions; compare them with package files and project target framework.

Create or select a development database and apply the documented schema/import process.

Configure local connection settings using placeholders from the README; never copy secrets into the report.

Restore backend dependencies and build the backend.

Install frontend dependencies using the lockfile-compatible command described by the repository.

Start the backend and record the actual address and port displayed by the application.

Start the frontend and record its actual address and port.

Open the API documentation if available, confirm the backend responds, and then test a frontend workflow.

Run automated tests and record the output before changing code.

Typical commands, to be run only in the correct project directories and adapted to the repository README:

```
dotnet restore
dotnet build
dotnet test

node -v
npm -v
npm install
npm run dev
```

Use the package manager and commands specified by the repository. If a lockfile exists, prefer the project's documented reproducible install process.

9.3 Common troubleshooting

Symptom	Checks to perform	Evidence to record
Frontend page does not load	Confirm the frontend command succeeded, the port is correct, and no build error remains.	Terminal output and actual local URL.
Frontend cannot retrieve data	Confirm backend is running, inspect browser network/console errors, check API base URL and CORS configuration.	Request URL, status code and redacted error.
Database connection fails	Check MySQL service, database name, user permissions and local connection settings.	Redacted connection configuration and server error.
API returns 404	Check the exact route in Swagger/controller definitions and confirm the correct host/port.	Actual route and response status.
API returns 500	Inspect server logs locally; do not expose stack traces to end users.	Redacted exception and reproduction steps.
EF Core runtime or build error	Check target framework and all EF/provider package versions as a compatible set.	dotnet list package and build output.

Data count is zero or unexpected	Confirm selected database/table, import mapping, and whether the app reads a different table.	SHOW TABLES, schema and count results.
Duplicate data after import	Stop repeated imports, back up the development database and identify duplicates precisely.	Source count, database count and correction plan.

10. Dependency injection, validation and exception handling

10.1 Dependency injection

In ASP.NET Core, dependency injection can register services such as the database context, repositories or application services so that controllers or endpoints receive the dependencies they require. The actual project structure must be inspected.

10.2 Input validation

Validation should be consistent across the UI and server. The frontend can help users correct mistakes early, while the backend must protect data integrity when requests arrive outside the normal UI. Validation.

10.3 Exceptions and logging

Expected validation failures should return understandable messages. Unexpected failures should be logged for developers and converted into safe responses for users. Logs should include enough context to diagnose an issue without exposing.

11. Testing and quality assurance

Automated tests are confirmed working. The exact framework, command, test count and pass/fail result are not yet provided, so these must be added from the actual test output.

11.1 Test levels

Unit tests: check small functions or services independently where the project has unit tests.

Integration tests: check communication between backend components, API routes and the database where supported.

UI/manual tests: confirm the visible user workflow from the browser.

Regression tests: repeat relevant existing tests after a change to ensure previously working features still work.

Data-quality tests: check row counts, required fields, ranges, relationships and duplicates.

Security/permission tests: check that protected operations reject unauthorised users.

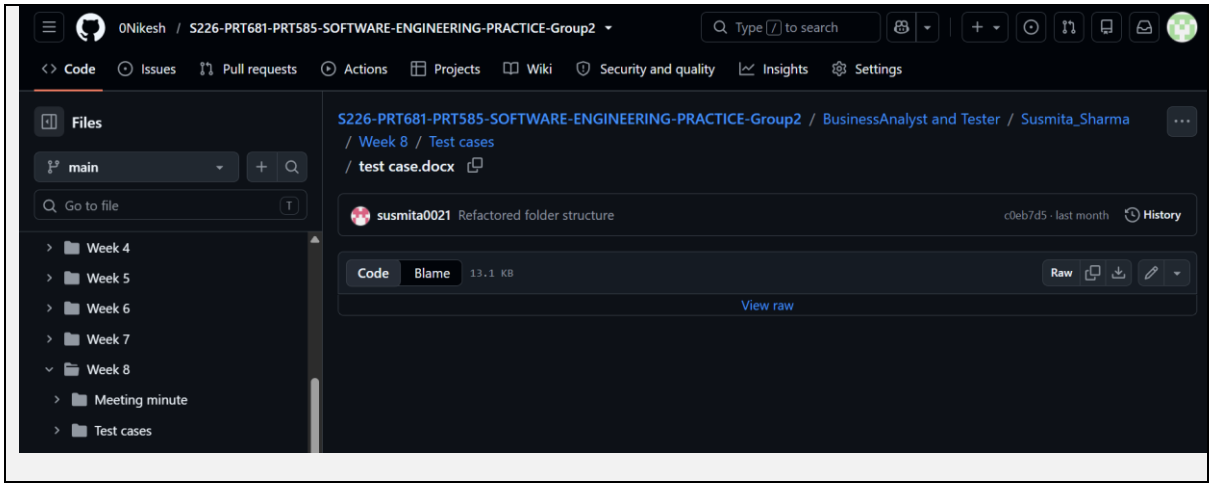
11.2 Suggested regression matrix

ID / feature	Test scenario	Expected result
AUTH-01 Registration	Submit valid test registration details.	Account is created and a clear outcome is shown.
AUTH-02 Login	Log in with valid test credentials.	User reaches the expected authenticated state.
AUTH-03 Invalid login	Use invalid test credentials.	Access is denied with a safe message.
DEST-01 Search	Search a known destination name.	Matching destinations appear.
DEST-02 Filters	Apply season and budget filters.	Results reflect the selected criteria.
SAVE-01 Saved destination	Save and revisit a destination.	Saved state remains according to intended persistence.
TRIP-01 Itinerary	Create/edit a test trip and itinerary entry.	Changes are displayed and linked correctly.
HOTEL-01 Planning record	Create a hotel planning record.	Planning record is stored as designed; no false supplier confirmation.
EXP-01 Rating	Submit a valid experience/rating.	Feedback appears with the correct rating scale.
ADMIN-01 CRUD	Admin adds, edits and removes a test destination.	Changes are applied and visible.
ADMIN-02 Permissions	Attempt admin operation as ordinary user.	Operation is denied if role protection is required.

DB-01 Connection	Read data through the application.	Expected records are returned from the configured source.

11.3 Recording defects

When a test fails, record the build/commit, test data, steps to reproduce, expected behaviour, actual behaviour, severity, screenshot or log, owner and status. Retest after a fix and retain the result. A screenshot.



12. Guide to adding a new feature

The following workflow is a reusable guide for adding a feature without damaging existing functionality. It is a recommended handover process; follow the actual team's review and branching policy where it differs.

Write the user problem and acceptance criteria in plain language.

Confirm the relevant team role and add the task to the actual project board or tracking method.

Review the existing screen, endpoint, entity, table and tests that the feature will affect.

Decide whether the change needs new fields, a new table, a relationship or a migration.

Update the ER diagram and data dictionary if the database design changes.

Implement server-side validation and permission checks for any data-changing operation.

Implement or update the backend endpoint and keep request/response fields consistent with the frontend.

Implement the React.js screen/component, including loading, success, empty and error states.

Add automated tests for normal cases, invalid input and relevant access rules.

Run build and regression tests; test the end-to-end workflow with safe test data.

Review the code diff, remove secrets, update documentation and capture evidence.

Commit and submit the change through the team's actual review process; update task status.

12.1 Acceptance checklist

Acceptance criteria are demonstrably met.

Data fields, API contract and schema changes are documented.

Validation and authorisation are tested on the server where relevant.

Automated tests pass and relevant manual regression checks are completed.

Database migration or setup changes can be reproduced safely.

Loading, empty and error states are understandable.

Documentation, ERD and screenshots match the new behaviour.

Git history and task records make the change traceable.

13. Individual contribution and evidence

13.1 Nikesh Shrestha – Data Analyst and DBA

The user's assigned role is Data Analyst and Database Administrator. The contribution should be presented through verifiable tasks and artefacts rather than broad claims. The project context includes selecting travel destination data for TripMate, reviewing data fields for destination discovery.

The most relevant DA/DBA responsibilities for this project are:

Understand the source dataset and map its fields to the destination information needed by the product.

Review data quality, missing values, inconsistent formats, duplicates and numeric ranges.

Help ensure that the MySQL schema supports the actual application workflows.

Validate imports by checking the correct database/table, row counts and sample records.

Use SQL to inspect data and verify data integrity, adapting queries to the real schema.

Maintain an ER diagram and data dictionary that match the current schema.

Communicate field names, types, nullability and relationships to the backend developer.

Document safe setup, backup and recovery practices for the database.

Help the team distinguish data that persists in MySQL from any data kept in browser storage.

Provide honest evidence of data and database tasks, including limitations and unfinished work.

Do not state that every item above was personally completed unless supported by the user's records. Once the ER diagram and CSV.

Recorded contribution. The Assessment 2 project progress report (18 September 2026) records Nikesh as the Data Analyst, with responsibility for data collection and data cleaning. The shared project repository is hosted under the 0Nikesh GitHub account and contains a "Nikesh Shrestha" folder, whose latest commit message is "Week 7 individual report".

Jira evidence. The Jira board shows a work item relating to data fields and data requirements, created on 11 September 2026 and assigned to Nikesh. It was still marked To Do and Unresolved when the evidence screenshot was taken.

Peer feedback received. Improve documentation of technical decisions; break large tasks into smaller manageable tasks; and conduct more frequent peer reviews.

13.2 Simran Chauhan – Frontend Developer and Report Analyst

Role. Simran Chauhan (S393695) was responsible for the React.js user interface and frontend functionality, and for supporting the group report, documentation and meeting records.

Research and learning. Simran developed her understanding of full-stack development, particularly how a React.js frontend communicates with an ASP.NET Core Web API through REST APIs. She also strengthened her knowledge of JWT-based authentication, database relationships, Entity Framework Core, API testing and Git-based development workflows, and applied these concepts to the TripMate application rather than learning them in isolation.

Implementation. The frontend work recorded in the progress report includes the login and registration screens, a dashboard, trip creation functionality and a basic itinerary viewer. Redux Toolkit is used for authentication and trip-related state management, and Axios interceptors handle JWT tokens and token expiration. The project screenshots show the home page with the destination search bar (where, dates and guests) and Popular Destinations cards, and a search results page with price range, travel type, rating and budget-tier filters, sorting and pagination. The GitHub commit "feat: added dashboard page for frontend" (32 files changed) is part of this work.

Project management, reporting and documentation. Simran kept her assigned tasks updated on Jira and recorded team activity in the repository. Two of her Jira work items were marked Done: "Finalise BA/tester documentation" (highest priority) and "Finalise report, references, screenshots and p…" (the title is truncated in the Jira screenshot). Her repository commits include "Minute Meeting" (16 September 2026), "Individual Timesheet Update", "Developers Meeting / Project Discuss" and "Report Analyst Meeting" (24 August 2026), and file uploads on 18 September 2026. Her individual timesheet is kept in the "Simran Chauhan" folder of the repository.

Challenges and solutions. Changes to API contracts could break frontend components that had already been built, so shared Swagger API documentation was used and changes were communicated before integration. CORS between the React frontend and the ASP.NET

backend was resolved by reviewing the backend configuration and testing client–server communication. Token expiration was handled with Axios interceptors.

Peer feedback received. Communicate task dependencies earlier; communicate through team meetings, team chat and class group discussion; provide regular updates when a task is delayed; and add more detailed descriptions to project-management cards.

Self-reflection. Simran identified three areas for personal improvement: time management (dividing large tasks into smaller ones and keeping to the schedule), technical confidence (understanding underlying concepts rather than relying on existing examples) and communication (clearer and more frequent updates on progress, problems and dependencies).

13.3 Bibek KC – Backend Developer and Automated Tester

Role. Bibek KC (S394325) was responsible for the ASP.NET Core APIs, backend integration and automated testing.

Recorded contribution. The progress report records the following backend work, which falls within Bibek's responsibility for the ASP.NET Core APIs: separate controllers, services, repositories and models; user registration and login using ASP.NET Identity and JWT authentication, with password hashing handled by Identity; CRUD APIs for trips and itinerary-related functionality; Entity Framework migrations; and Swagger configured for API documentation and testing. At the report date the backend was approximately 75% complete and authentication was complete.

Repository evidence. The repository contains a "Bibek KC" folder, whose latest commit message is "feat: added the meeting of week07". The Software Development folder includes the commit "feat: automated tested for login and registration", which supports the automated testing role.

Peer feedback received. Maintain consistent Git commit messages; test features before marking tasks as completed; and share API or UI changes with the team before integration.

13.4 Susmita Sharma – Business Analyst and Manual Tester

Role. Susmita Sharma (S394867) was responsible for requirements, user workflows, manual testing, validation and quality. The progress report lists her under QA/Tester responsibilities for testing, validation and quality.

Recorded contribution. Susmita gathered and analysed project requirements, identified business needs and helped define the system objectives, and prepared and documented the functional and non-functional requirements.

Evidence. The repository contains a "Susmita Sharma" folder, whose latest commit message is "Added test cases, test results and requirement review". On Jira, Susmita is the assignee or reporter on many of the Done work items created on 11 September 2026, including documentation and testing tasks, and she reported the highest-priority item "Finalise BA/tester documentation".

13.5 Evidence register

The table below lists the evidence cited in this section. Screenshots are included in Appendices A to C of the Assessment 2 project progress report.

Evidence	Location	Relates to
Group GitHub repository (86 commits, one branch at the time of the screenshot)	https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2	All members
Simran Chauhan commit history	https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2/commits/main/?author=simran22chauhan	Simran Chauhan
Frontend commit "feat: added dashboard page for frontend" (299324a)	GitHub repository, main branch	Simran Chauhan
Jira work items (Done)	https://simran22chauhan2000.atlassian.net/jira/for-you?tab=workedon	Simran Chauhan
Jira work item list (55 items at the time of the screenshot)	https://sharmasusmits04.atlassian.net/issues?filter=-4&jql=ORDER%20BY%20reporter%20ASC%2C%20created%20DESC	All members
Frontend screenshots: home page and destination search results	Progress report, Appendix A	Simran Chauhan
Individual timesheet	Repository folder "Simran Chauhan"	Simran Chauhan
Repository folders with individual reports and meeting notes	Folders "Bibek KC", "Nikesh Shrestha", "Susmita Sharma", "Software Development" and "Minute Meeting"	Bibek KC, Nikesh Shrestha, Susmita Sharma

13.6 Team contribution summary

TripMate is a four-person project with complementary roles. The backend developer and automated tester is responsible for backend logic and automated test work; the frontend developer and report analyst supports the React.js user interface and reporting; the DA/DBA

supports data quality and database structure; and the business analyst and manual tester supports requirements, acceptance criteria and manual testing.

14. Time management and team collaboration

A handover report should show how work was planned, completed, checked and carried forward. The project's earlier plan was organised into short weekly phases, including authentication and database setup.

A useful weekly record contains the date or week, objective, attendees, completed work, outstanding actions, next-period work and issues raised. A personal timesheet should connect hours to actual tasks.

14.1 Collaboration practices

Agree on database field names and API contracts before implementing dependent screens.

Share schema or ERD changes with the backend developer before they are merged.

Use one agreed source of truth for the dataset and database setup instructions.

Raise import problems early, including unexpected row counts or duplicate data.

Use short, specific issue reports with reproduction steps and evidence.

Review the user manual and presentation against the running build before describing a feature as complete.

Keep the handover report updated when an endpoint, schema, dependency or startup command changes.

15. Limitations, risks and future work

15.1 Current boundaries

The following boundaries are supported by the material currently available: application URLs were not provided; the exact dependency versions are not yet confirmed; the current ER diagram and CSV have not yet been attached to this update; and.

15.2 Risks and mitigations

Risk	Why it matters	Practical mitigation
ERD does not match code	Developers may misunderstand relationships or break queries.	Compare ERD, MySQL schema and entity classes; update together.
Data imported into wrong table	UI can show zero or inconsistent records even	Confirm database/table and compare row counts and sample rows.

	when an import message says success.	
Repeated CSV import	Duplicate destinations may appear in search or reports.	Back up first; identify duplicates using stable keys and correct them safely.
Version mismatch	Build/runtime errors can block startup or data access.	Record package versions and keep EF Core provider versions compatible.
Weak permission checks	A normal user could change shared destination records.	Test admin operations with both admin and ordinary test accounts.
Browser-storage assumptions	Data may not sync across browsers/devices or survive clearing site data.	Document the actual persistence location and move to server-side storage only if required.
Planning record mistaken for confirmed booking	Users could believe a hotel is reserved when it is not.	Use clear UI wording and implement supplier integration only as a separate, tested feature.
No reproducible setup	Future developers cannot run or validate the project.	Maintain README, schema scripts, environment variable names and test commands.

15.3 Future improvements

Possible next steps should be prioritised by the team and not described as already implemented. These may include confirming and documenting the final deployment state, improving automated test coverage, reviewing role-based access controls, monitoring performance and database errors, improving accessibility and.

15.4 Handover notes for the next team

Start from the current repository and verify the branch and commit before setup.

Use the actual README, project files and schema rather than copying generic commands blindly.

Keep local credentials private and use test data for demonstrations.

Confirm the ERD matches MySQL and the backend entities.

Record actual frontend, backend and Swagger URLs after starting the application.

Run the automated test suite and save its output.

Confirm which features persist in MySQL and which, if any, use browser storage.

Preserve the distinction between hotel planning records and confirmed external reservations.

Maintain a list of known issues with owner, severity, reproduction steps and status.

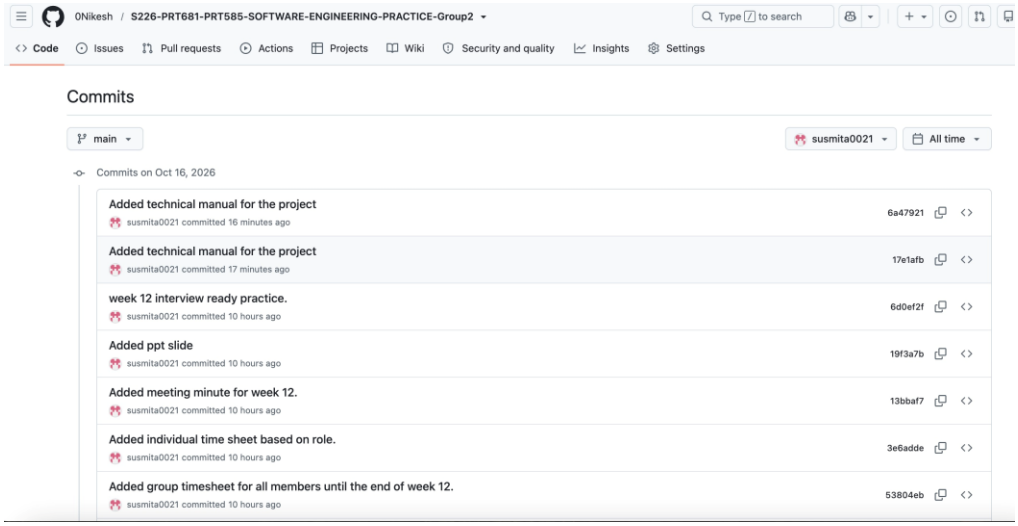
Appendix A – Screenshot and evidence register

Figure	Evidence requested	What it should demonstrate
A1	Home/Explore page	Application loads and destination discovery is visible.
A2	Registration and login	Confirmed account workflow using test data.
A3	Search and filtering	A query and filters change the result set.
A4	Destination details	Actual fields available for planning.
A5	Saved destinations	Save action and saved list.
A6	Trip and itinerary	Create/edit workflow and resulting entries.
A7	Hotel planning record	Planning details without implying supplier confirmation.
A8	Experiences and ratings	Submission and display of feedback.
A9	Admin destination CRUD	Add, edit and remove screens, including validation if available.
A10	MySQL schema and query output	Actual table names, sample rows and validation checks.
A11	ER diagram	Current relationships and keys.
A12	Automated test output	Real command and pass/fail results.

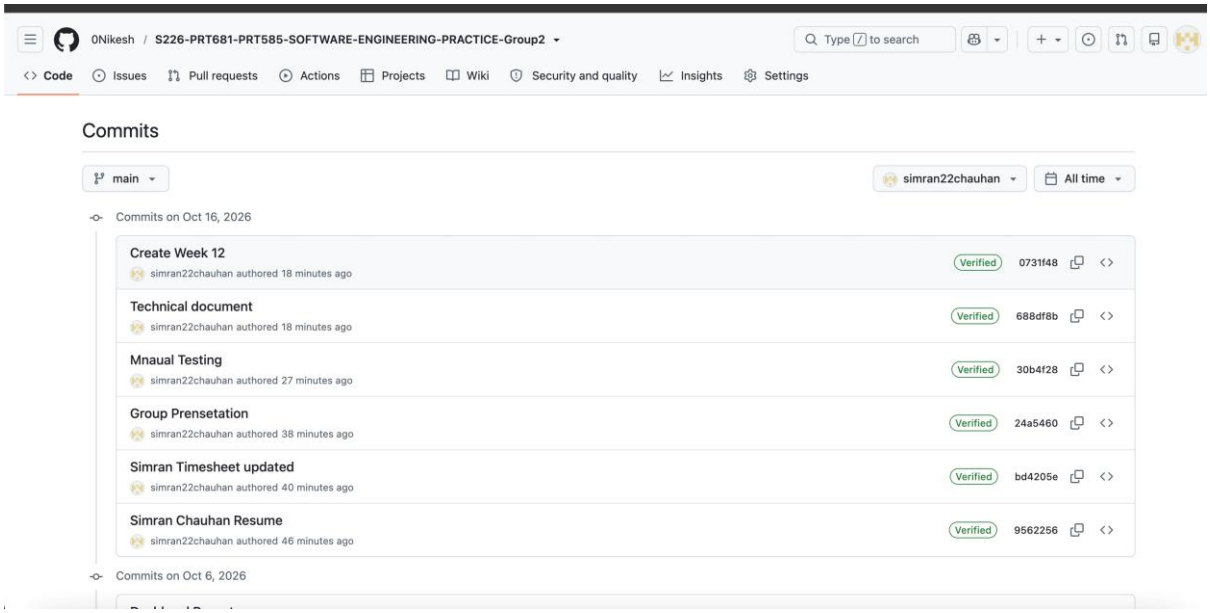
A13	Dependency versions	Frontend package versions and .NET/EF package list.
A14	Repository/task evidence	Actual commit and board/task if used.

Screenshots

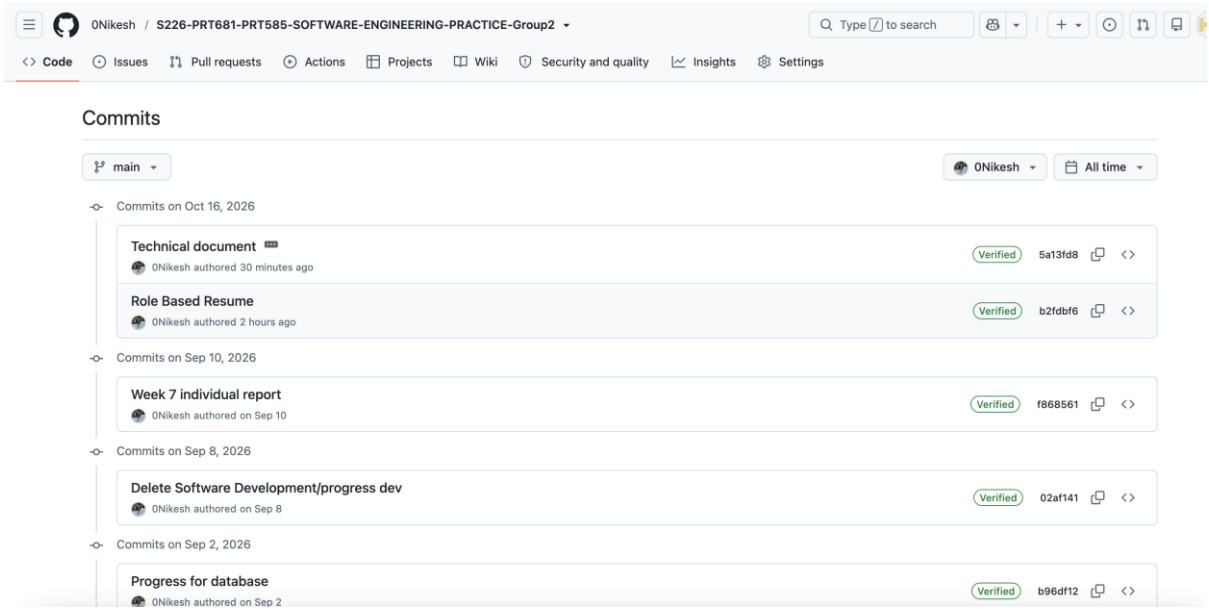
1. **Guit Hub Commit (https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2/commits/main/?author=simran22chauhan Susmita Sharma)**



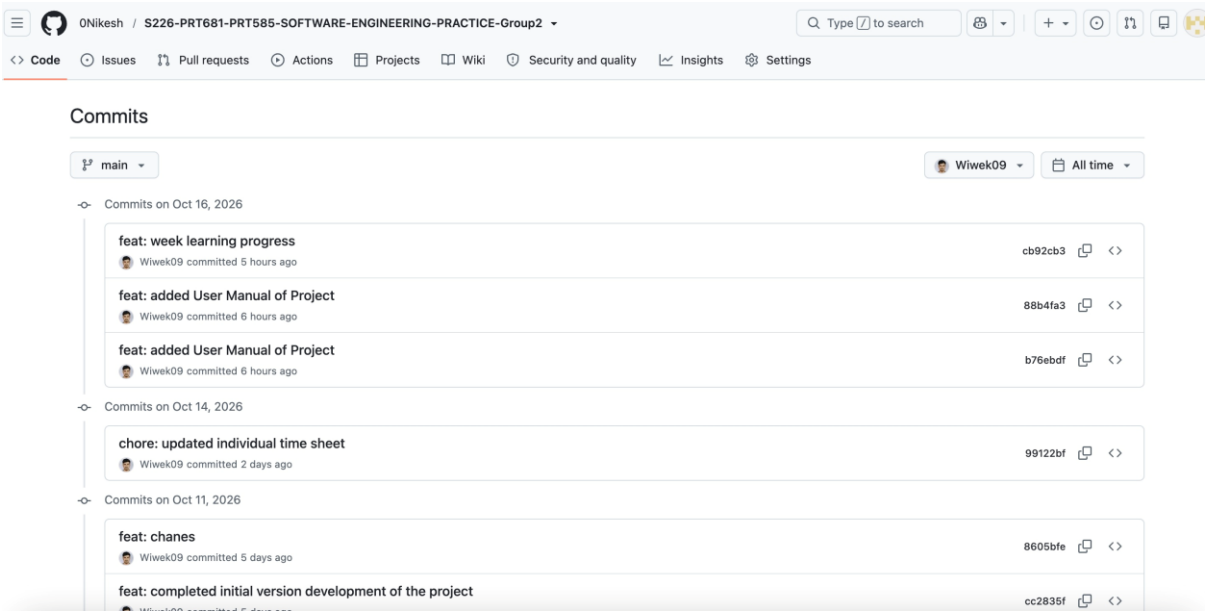
Simran Chauhan



Nikesh

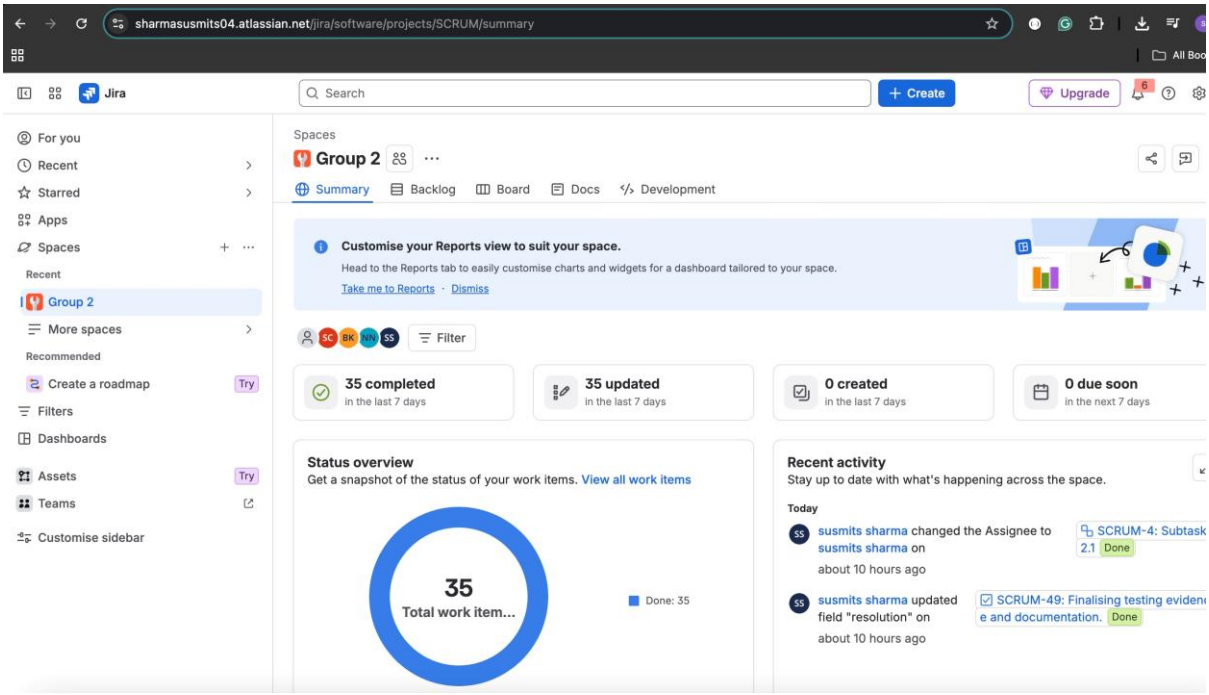


Bibek Kc

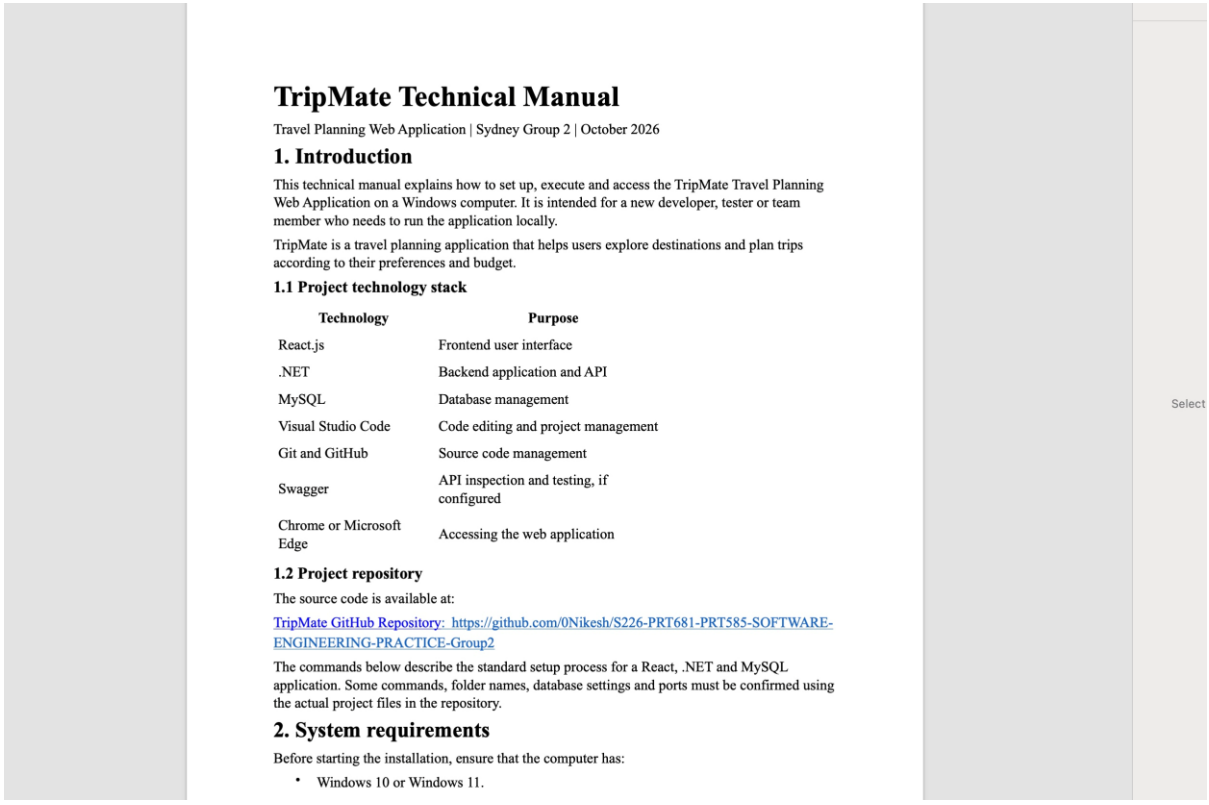


2. Jira

(https://sharmasusmits04.atlassian.net/jira/software/projects/SCRUM/summary)



3. Manual Testing



TripMate Technical Manual

Travel Planning Web Application | Sydney Group 2 | October 2026

1. Introduction

This technical manual explains how to set up, execute and access the TripMate Travel Planning Web Application on a Windows computer. It is intended for a new developer, tester or team member who needs to run the application locally.

TripMate is a travel planning application that helps users explore destinations and plan trips according to their preferences and budget.

1.1 Project technology stack

Technology	Purpose
React.js	Frontend user interface
.NET	Backend application and API
MySQL	Database management
Visual Studio Code	Code editing and project management
Git and GitHub	Source code management
Swagger	API inspection and testing, if configured
Chrome or Microsoft Edge	Accessing the web application

1.2 Project repository

The source code is available at:

TripMate GitHub Repository: https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2

The commands below describe the standard setup process for a React, .NET and MySQL application. Some commands, folder names, database settings and ports must be confirmed using the actual project files in the repository.

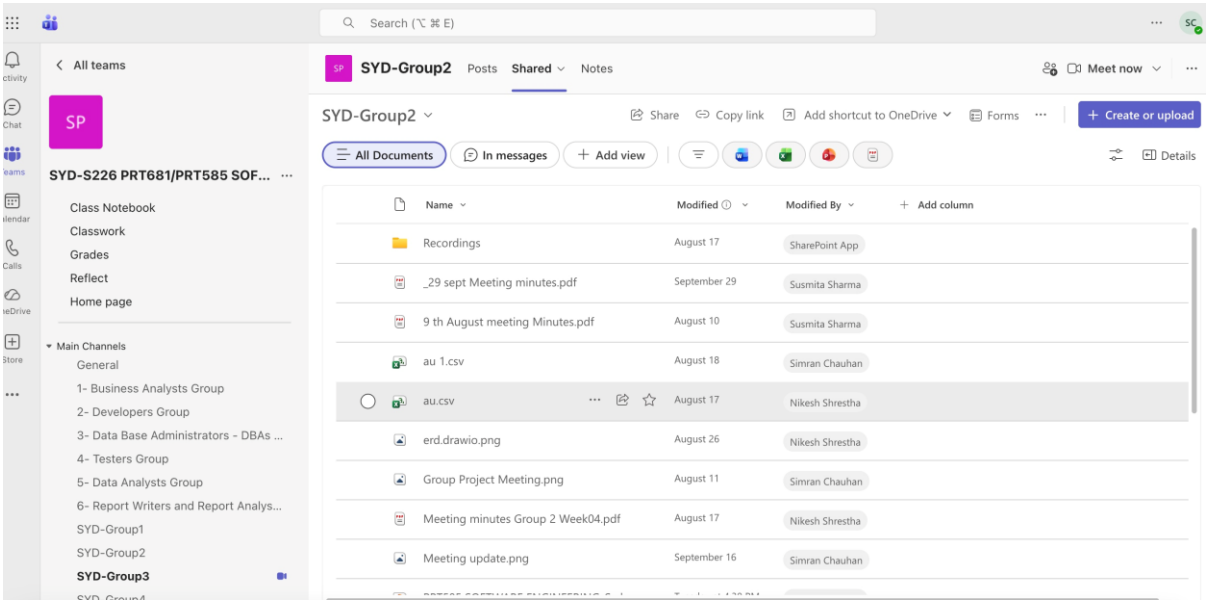
2. System requirements

Before starting the installation, ensure that the computer has:

- Windows 10 or Windows 11.

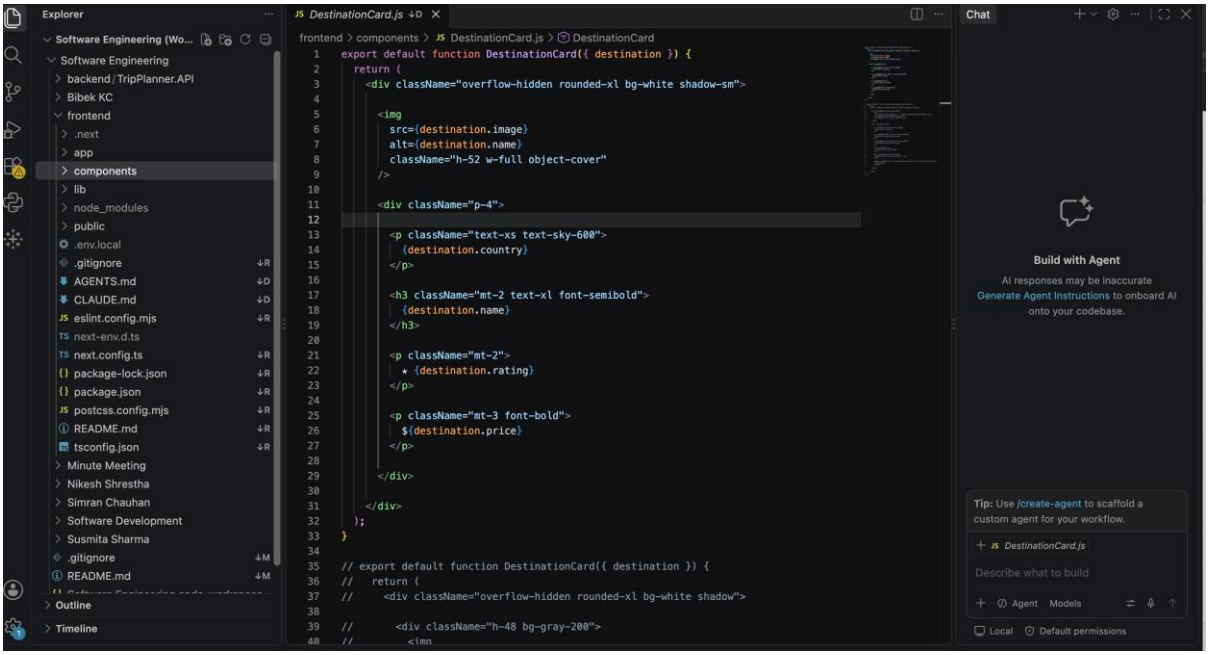
Select

4. Team Channel (Connect with team and discussion) https://teams.cloud.microsoft/

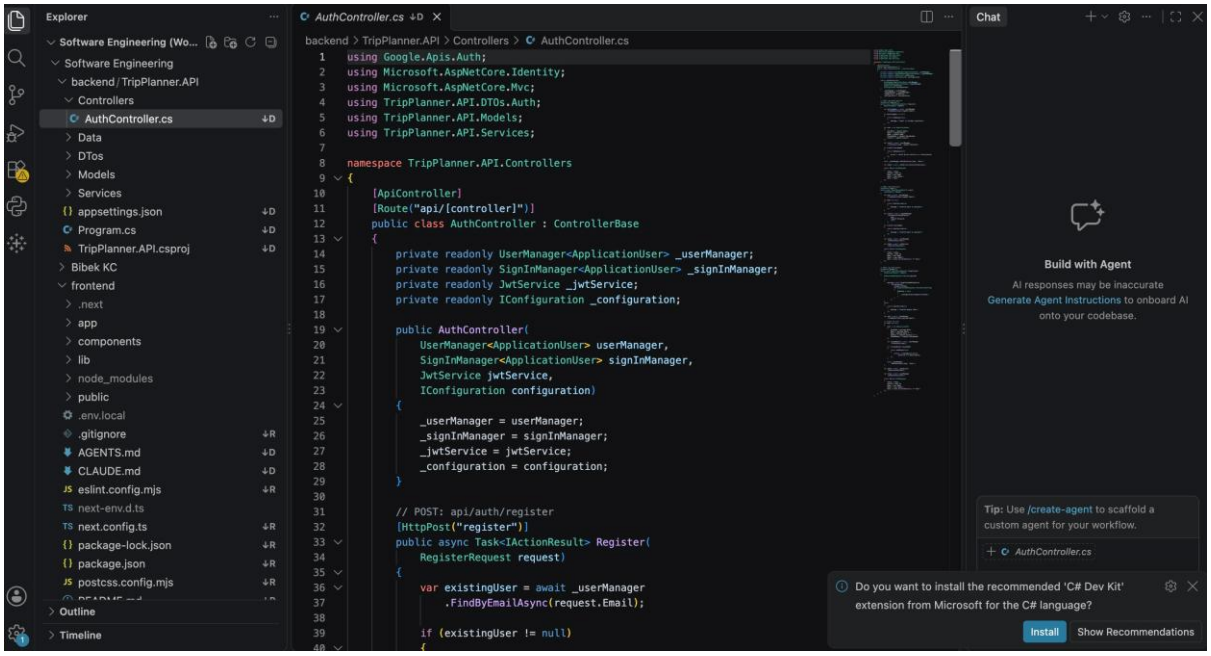


5. Coding

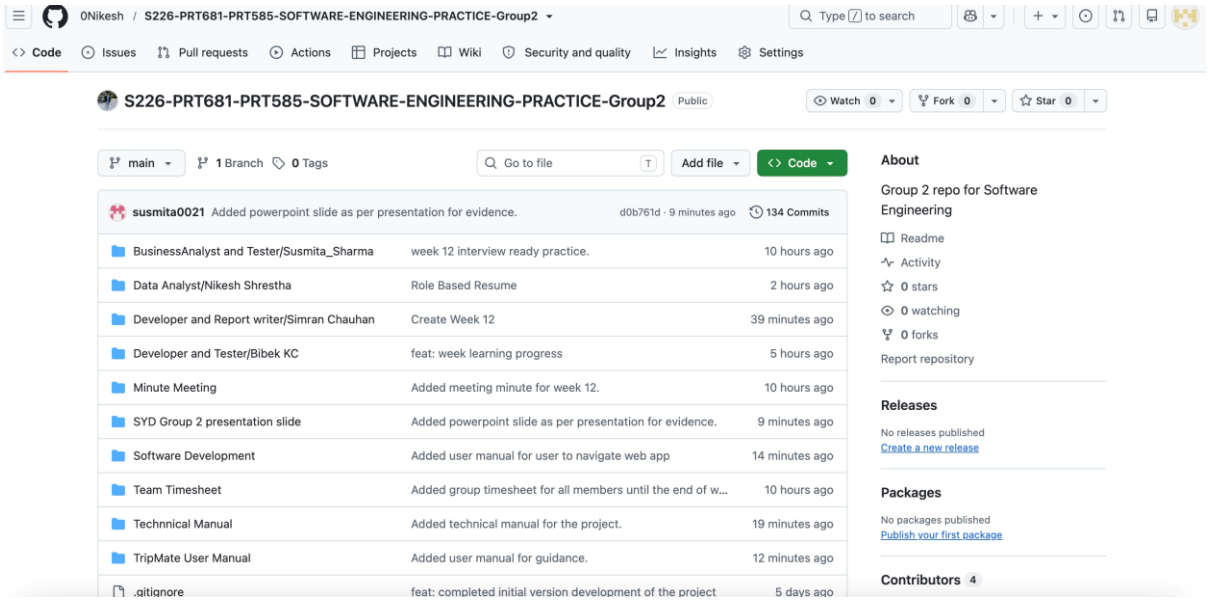
Frontend(Simran Chauhan)



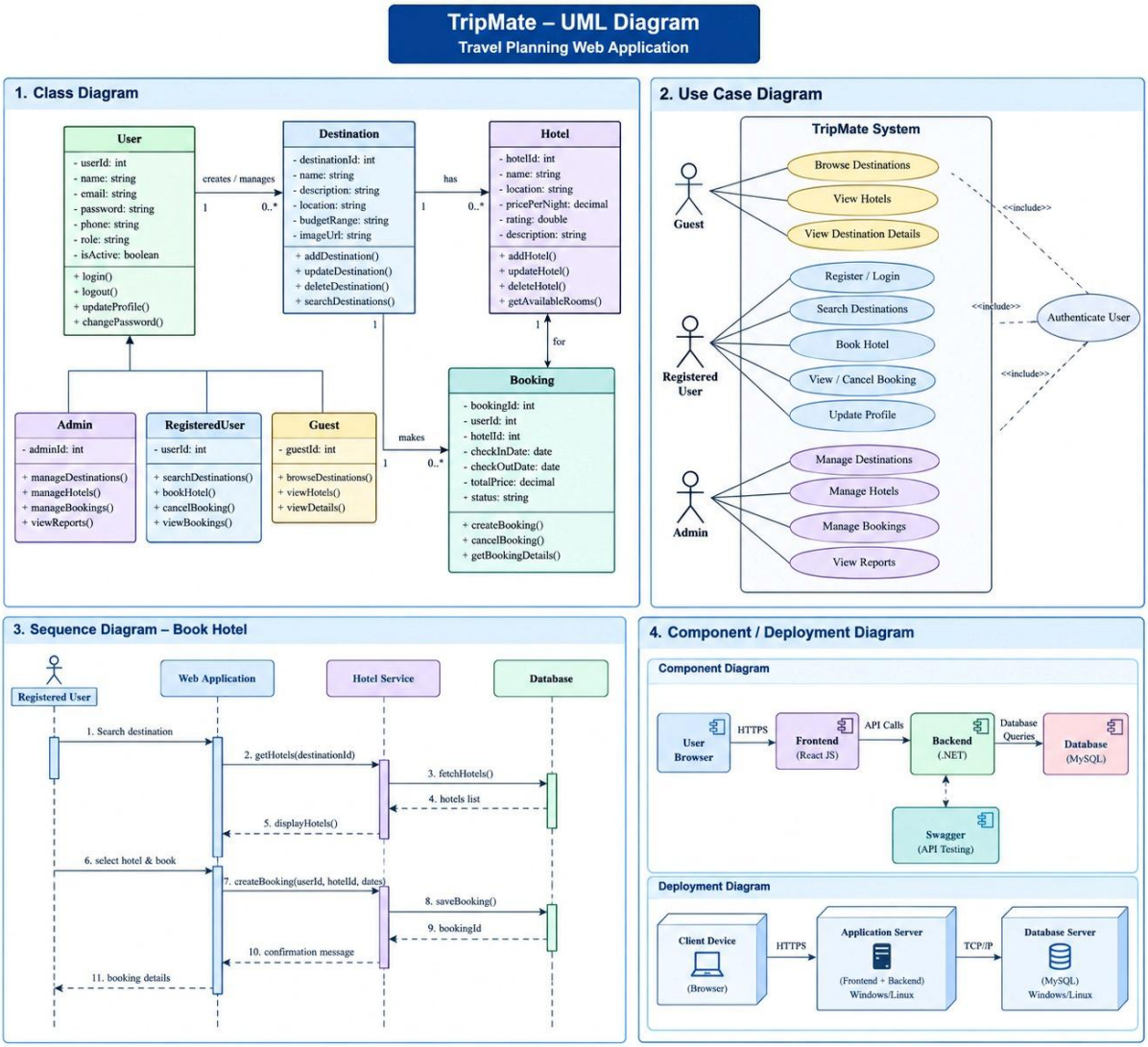
Backend (Bibek KC)



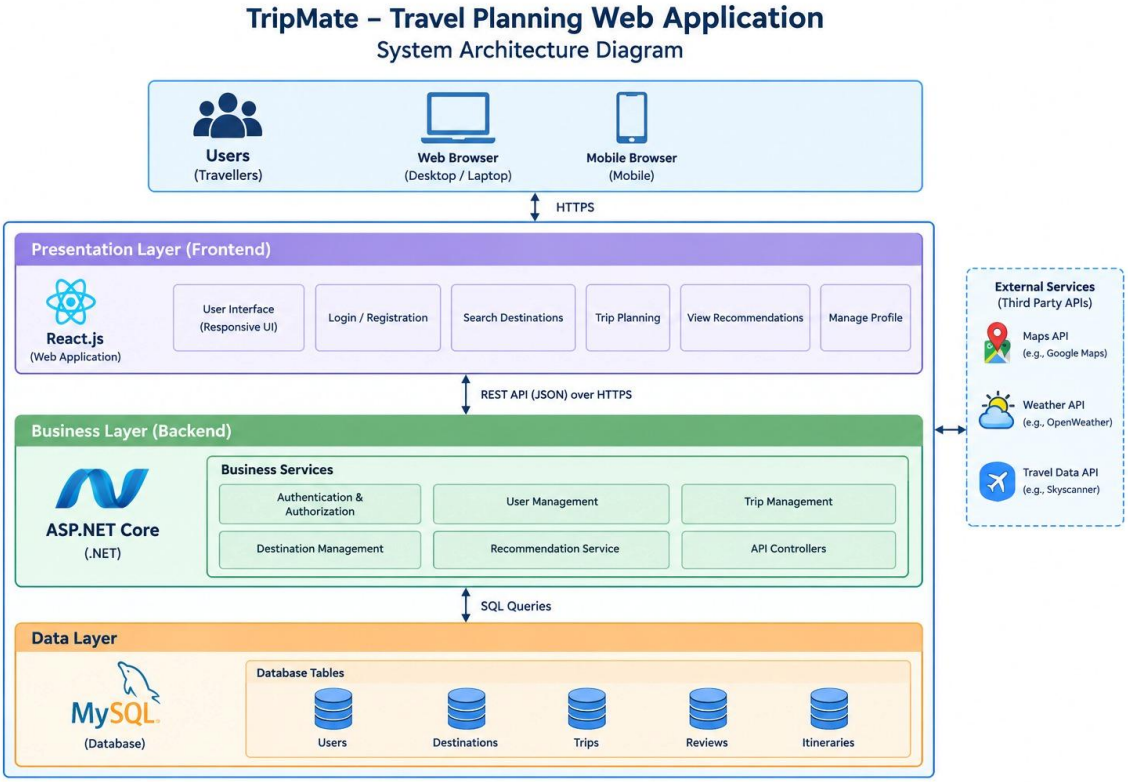
6.Guit hub Main Branch



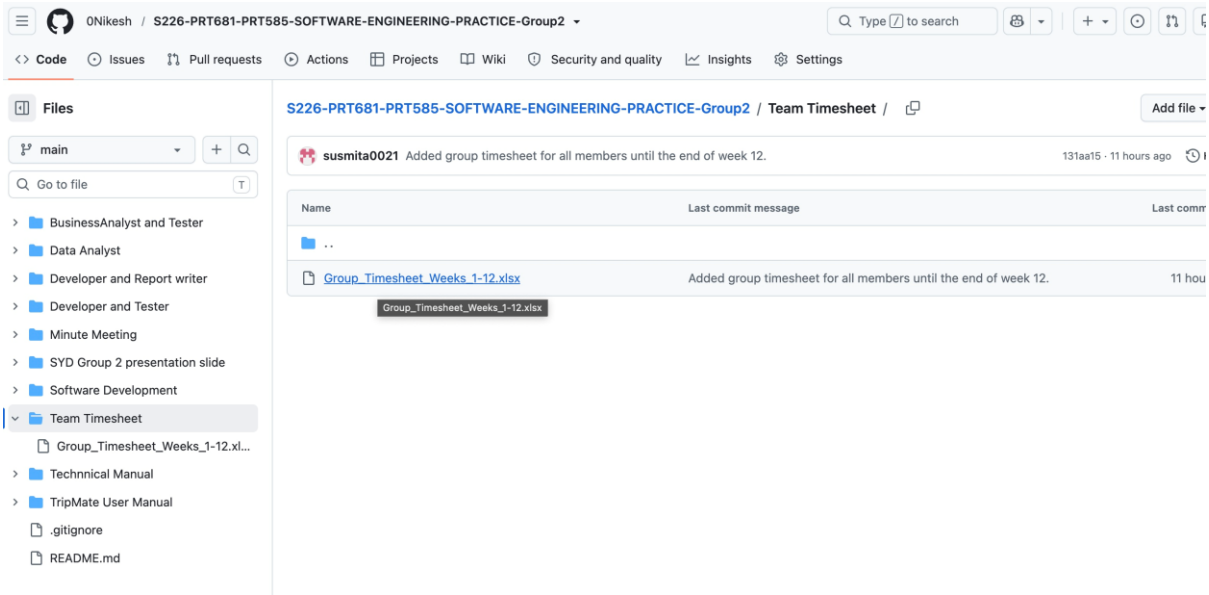
Tripmate UML Diagram



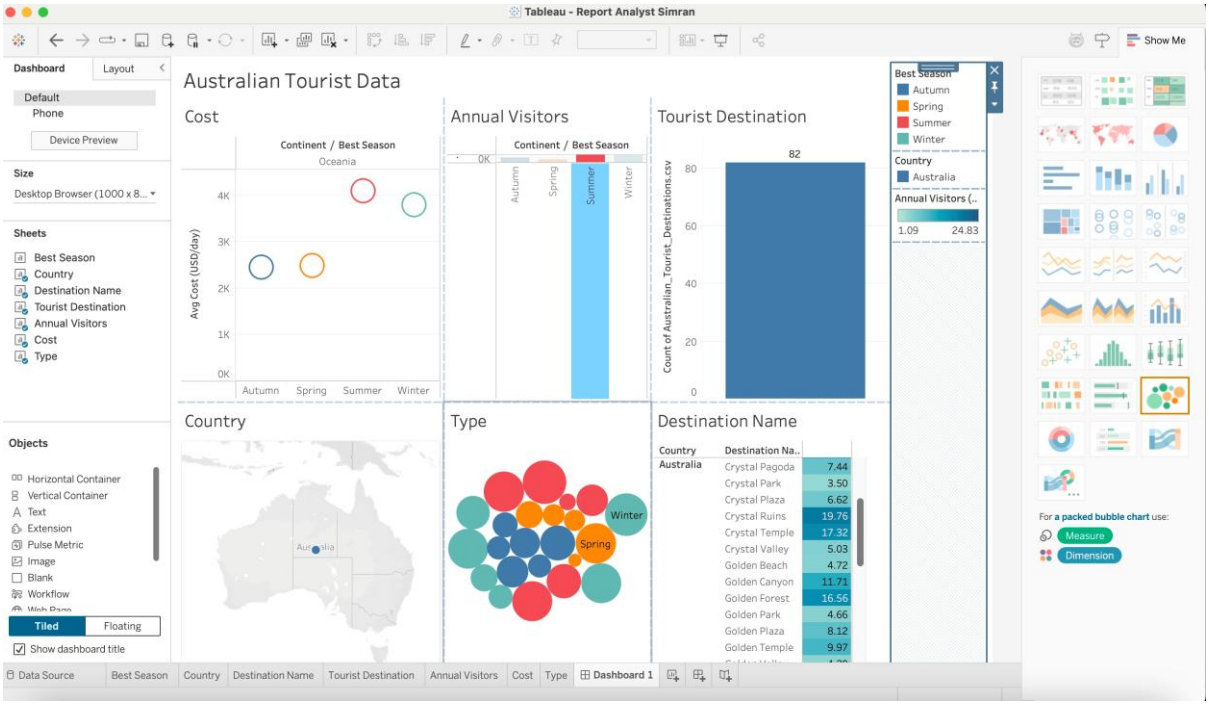
System Architecture



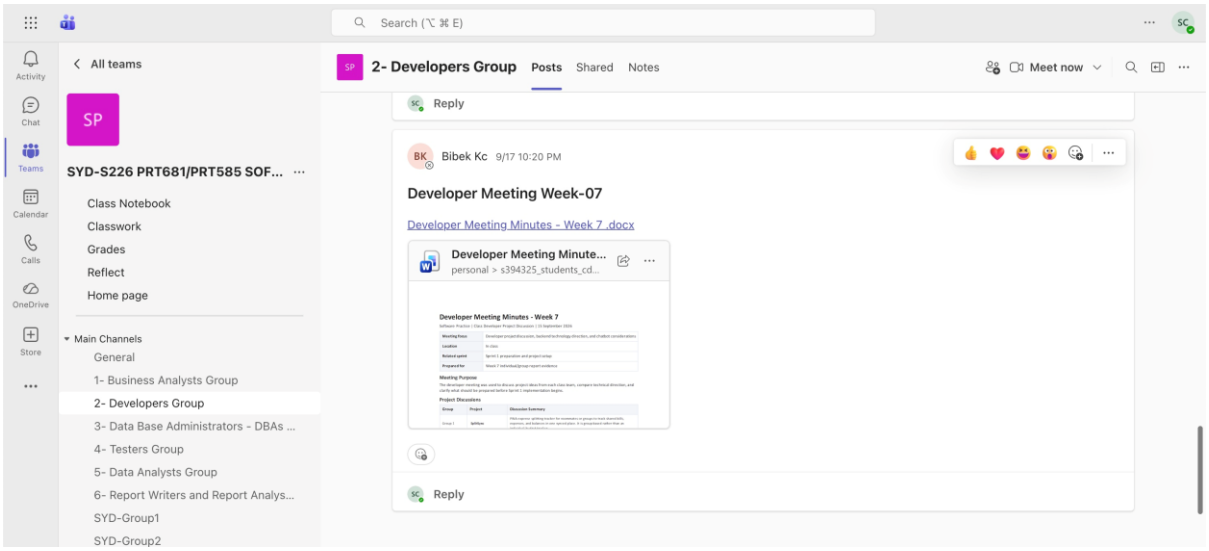
Group Timesheet



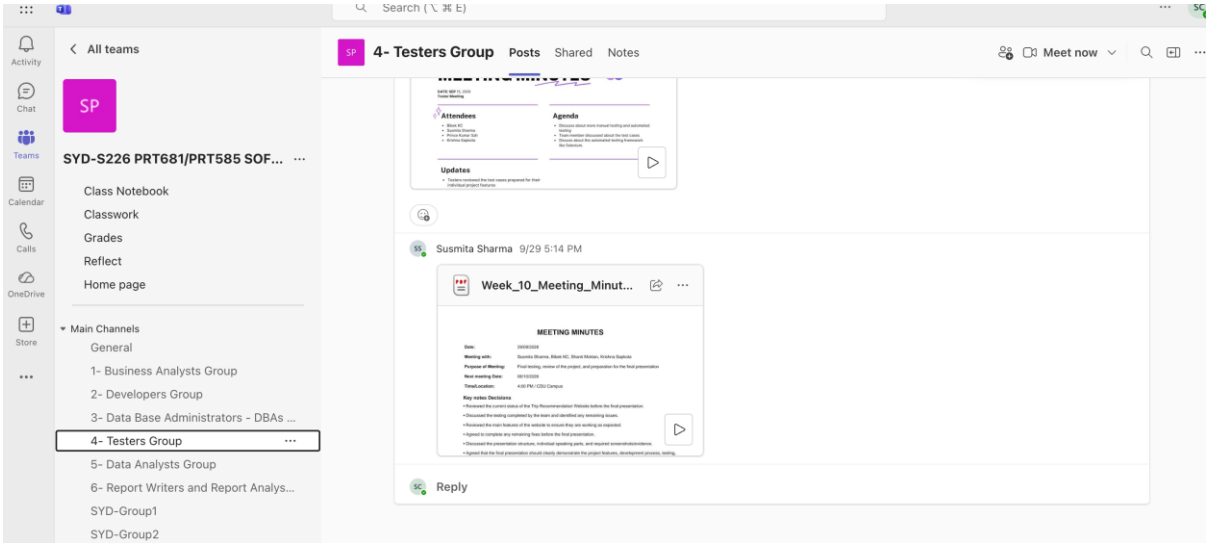
7.Report Analyst Simran Chauhan **Tableau Dashboard**

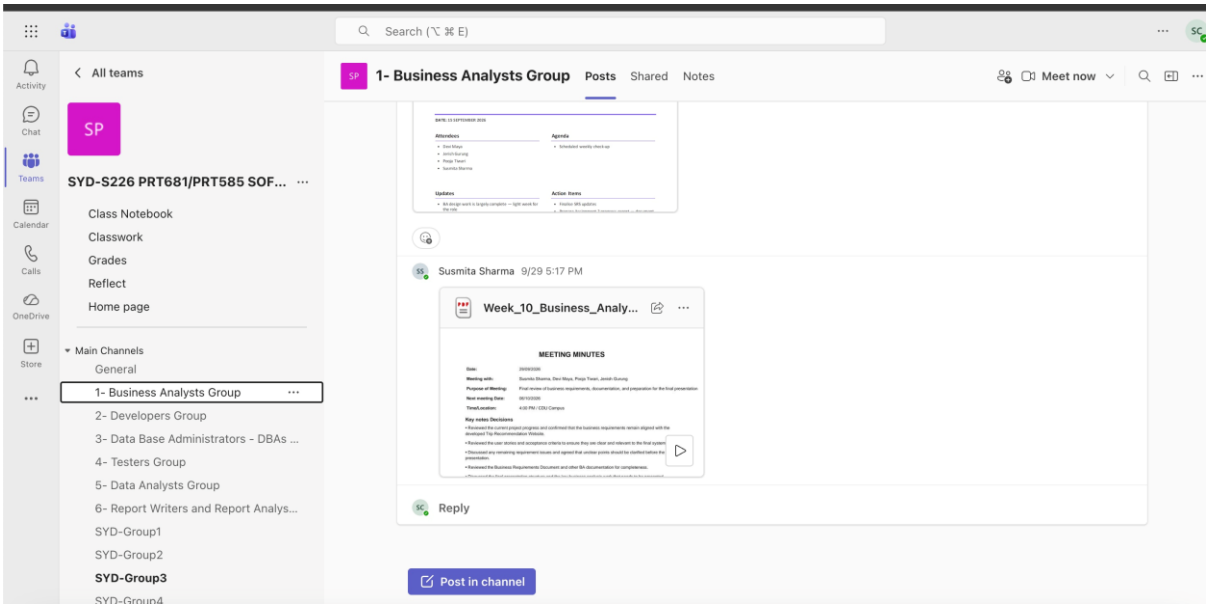


Developer Group(Simran Chauhan and Bibek Kc)

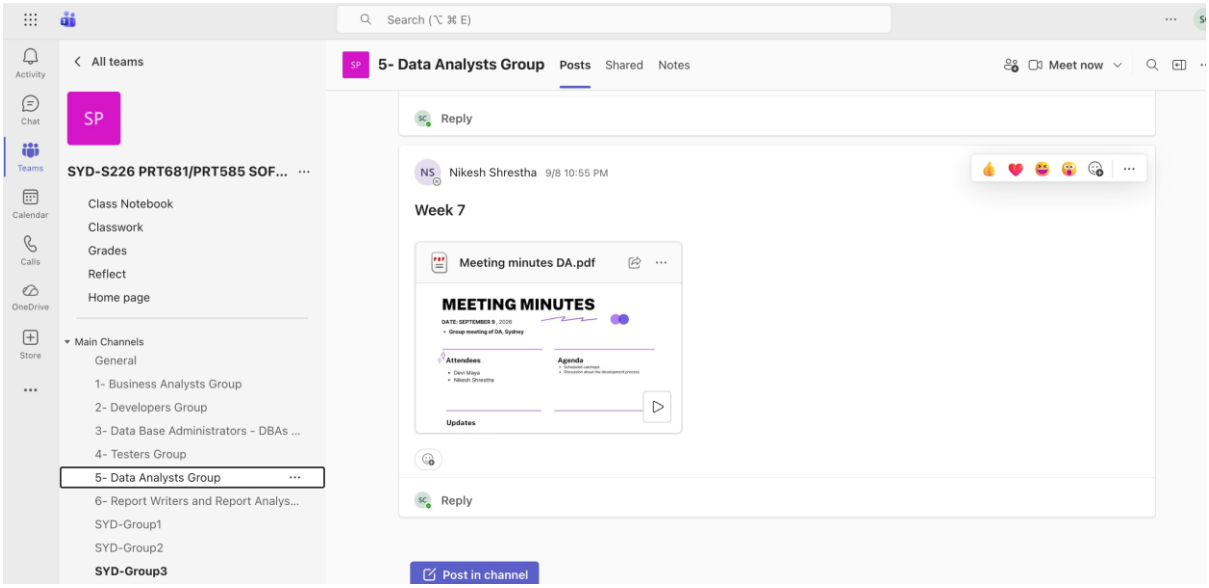


Business Analyst and tester(Susmita Sharma)





Nikesh (Data Analyst)



All Links

1. https://sharmasusmits04.atlassian.net/jira/software/projects/SCRUM/summary
2. https://github.com/0Nikesh/S226-PRT681-PRT585-SOFTWARE-ENGINEERING-PRACTICE-Group2
3. https://teams.cloud.microsoft/